Airbnb New User Bookings

Data Mining and Machine Learning Project

Reza Almassi & Jad Mahmoud

Supervisor: Prof. Francesco Marceloni

University of Pisa

Academic Year 2024/2025

Abstract

This report presents a comprehensive machine learning study on the **Airbnb New User Bookings** prediction problem, originally posed as a Kaggle recruiting competition. The task is a 12-class classification challenge: given demographic and behavioural data about a new Airbnb user, predict the country of their first booking destination, where the majority of users either do not book (`NDF`) or book within the United States (`US`), creating a heavily imbalanced label distribution.

The study begins with an in-depth exploratory data analysis of three data sources — training users, test users, and raw web-session logs comprising over 10.5 million records — followed by a rigorous preprocessing pipeline that addresses missing values, erroneous age entries, and heterogeneous categorical features. An extensive feature engineering stage aggregates session-level clickstream signals into per-user statistics, including action counts, duration metrics, pause patterns, and device category flags, which are subsequently merged with the user demographic table to form the final feature matrix.

A range of classification models is trained and evaluated, from simple baselines (Logistic Regression, Decision Tree) to high-capacity gradient-boosted ensembles (XGBoost, LightGBM, CatBoost). To tackle the inherent complexity of the 12-class problem, a **hierarchical three-stage architecture** is proposed, decomposing the prediction into sequential binary and multi-class sub-problems. Class imbalance is addressed through SMOTE-based oversampling and balanced bootstrap strategies. The best individual models are further combined via **ensemble learning**, and hyperparameters are optimised using Optuna. All models are assessed using Stratified K-Fold cross-validation and a suite of metrics including accuracy, macro F1, weighted F1, and the competition's primary metric, **NDCG@5**.

The full source code, notebooks, and data pipeline for this project are publicly available at:

https://github.com/Rezacs/Airbnb

Contents

Chapter 1

Introduction

1.1 Context and Motivation

The rapid growth of online travel platforms has generated vast amounts of user behavioural data, offering an unprecedented opportunity to understand and anticipate customer decisions. Airbnb, one of the world's leading home-sharing marketplaces, operates in more than 220 countries and processes millions of bookings annually. A key challenge for such platforms is predicting *where* a new user will make their first booking, since early personalisation of the user experience can significantly increase conversion rates and overall customer satisfaction.

This project is framed around the **Airbnb New User Bookings** Kaggle competition[1], which was originally released as a recruiting challenge. The competition provides real-world anonymised data on Airbnb users and their web-session activity, making it an ideal benchmark for exploring classification techniques on highly imbalanced, multi-class tabular data.

1.2 Problem Statement

Given a set of demographic and behavioural features about a new Airbnb user, the task is to predict in which country that user will make their first booking. The target variable consists of **12 possible destination classes**: `NDF` (no destination found — the user did not book), `US`, `other`, `FR`, `IT`, `GB`, `ES`, `CA`, `DE`, `NL`, `AU`, and `PT`.

The problem presents several inherent challenges:

- **Severe class imbalance** — the majority of users fall into the `NDF` or `US` class, while minority destination classes (e.g., `AU`, `PT`) are sparsely represented.
- **High proportion of missing values** — both the user demographic table and the session log contain substantial missing data that must be handled carefully.

[1] https://www.kaggle.com/c/airbnb-recruiting-new-user-bookings

- **Heterogeneous feature types** — the dataset combines continuous, categorical, and datetime features, as well as aggregated session-level signals derived from raw clickstream data.
- **Evaluation via NDCG@5** — the competition scores predictions using Normalised Discounted Cumulative Gain at rank 5, rewarding models that rank the correct destination highly rather than requiring an exact single prediction.

1.3 Project Objectives

The primary objectives of this project are:

1. To perform thorough **exploratory data analysis** on the Airbnb user and session datasets, uncovering patterns in user demographics, temporal behaviour, and session activity.
2. To design and implement a **comprehensive preprocessing and feature engineering** pipeline, including missing-value imputation, age cleaning, datetime decomposition, and session-level aggregation.
3. To train and compare multiple **classification models** — ranging from simple baselines (Logistic Regression, Decision Tree) to gradient-boosted ensembles (XGBoost, LightGBM, CatBoost).
4. To investigate a **hierarchical three-stage modelling strategy** that decomposes the 12-class problem into a sequence of binary and multi-class sub-problems.
5. To address **class imbalance** through sampling strategies (SMOTE, balanced bootstrap) and class-weight adjustments.
6. To **optimise and ensemble** the best-performing models, evaluating final performance using NDCG@5 and auxiliary metrics (accuracy, macro F1, weighted F1, top-5 accuracy).

Chapter 2

Dataset Description

2.1 Source and Context

The data used in this project originates from the **Airbnb New User Bookings** Kaggle competition, which was released as part of Airbnb's recruiting initiative. The competition challenges participants to predict in which country a new Airbnb user will make their first booking, based on demographic information and web-session activity logs. The full competition description, rules, and data files are publicly available at:

https://www.kaggle.com/c/airbnb-recruiting-new-user-bookings

All data is fully anonymised. Users are identified by randomly generated alphanumeric IDs, and no personally identifiable information is exposed. The data covers Airbnb user registrations up to June 2014, with session logs recorded during the same period. The competition was officially scored using **Normalised Discounted Cumulative Gain at rank 5 (NDCG@5)**, which rewards models that place the correct destination country within the top-5 predictions, with higher weight given to earlier ranks.

2.2 Files Overview

The dataset consists of three primary CSV files:

- `train_users_2.csv` — The training set containing **213,451 users**, each associated with a known `country_destination` label. This is the file used to train and evaluate all models.
- `test_users.csv` — The test set containing **62,096 users** without a known destination label. This file was used for final Kaggle submission. In total, the users table covers **275,547 unique users** across both splits.
- `sessions.csv` — A raw web-session log containing **10,567,737 recorded actions** across **135,483 unique users**. Each row represents a single user

action, capturing the type of interaction (e.g., click, view, submit), the device used, and the time elapsed. Notably, not all users in the users table have a corresponding session record — only approximately 49% of total users appear in the sessions data.

Table 2.1: Summary of the three dataset files.

File	Rows	Columns	Description
`train_users_2.csv`	213,451	16	Labelled user records
`test_users.csv`	62,096	15	Unlabelled user records
`sessions.csv`	10,567,737	6	Raw clickstream logs

2.3 Feature Descriptions

2.3.1 Users Table Features

The users table (`train_users_2.csv` and `test_users.csv`) contains 15–16 columns describing each user's demographic background, account registration details, and acquisition channel. Table 2.2 provides a complete description of each feature.

2.3.2 Sessions Table Features

The sessions table (`sessions.csv`) captures raw clickstream interactions at the action level. Each row corresponds to a single user action, and multiple rows may exist per user. Table 2.3 describes the six columns in this file.

2.4 Target Variable Distribution

The target variable `country_destination` takes one of 12 discrete values. Table 2.4 and Figure 2.1 reveal a heavily skewed distribution that poses a significant modelling challenge.

The class distribution reveals three important observations:

1. **Majority class dominance** — `NDF` alone accounts for over 58% of all training labels, meaning the majority of new users do not complete any booking at all. This is the single largest challenge of the task.

2. **US concentration** — Among users who *do* book, the vast majority (approximately 70% of bookers) travel within the United States, reflecting both Airbnb's US-heavy user base during this period and the domestic nature of much early platform usage.

Table 2.2: Feature descriptions for the users table.

Feature	Type	Description
`id`	Categorical	Unique anonymised user identifier
`date_account_created`	Date	Date on which the user created their account
`timestamp_first_active`	Datetime	Timestamp of the user's very first activity on the platform
`date_first_booking`	Date	Date of the user's first booking (NaN if no booking)
`gender`	Categorical	Self-reported gender: `MALE`, `FEMALE`, `OTHER`, `-unknown-`
`age`	Continuous	Self-reported age (contains outliers and erroneous year values)
`signup_method`	Categorical	Registration method: `basic`, `facebook`, `google`
`signup_flow`	Integer	Page variant used during registration
`language`	Categorical	Preferred interface language (ISO 639-1 code)
`affiliate_channel`	Categorical	Acquisition channel (e.g., `direct`, `sem-brand`)
`affiliate_provider`	Categorical	Traffic source (e.g., `google`, `facebook`, `direct`)
`first_affiliate_tracked`	Categorical	First marketing touch tracked before registration
`signup_app`	Categorical	Application used for sign-up: `Web`, `iOS`, `Android`, `Moweb`
`first_device_type`	Categorical	Device type at first activity (e.g., `Mac Desktop`, `iPhone`)
`first_browser`	Categorical	Browser used at first activity (e.g., `Chrome`, `Safari`)
`country_destination`	Categorical	**Target variable** — the country of first booking (train only)

Table 2.3: Feature descriptions for the sessions table.

Feature	Type	Description
`user_id`	Categorical	Foreign key linking to the users table
`action`	Categorical	Specific action performed (e.g., `search_results`, `lookup`)
`action_type`	Categorical	Category of action (e.g., `click`, `view`, `data`)
`action_detail`	Categorical	Detailed description of the action
`device_type`	Categorical	Device used for the action (e.g., `Mac Desktop`, `iPhone`)
`secs_elapsed`	Continuous	Time elapsed (in seconds) spent on the action

3. **Sparse minority classes** — Destinations such as `PT` (0.10%), `AU` (0.25%), and `NL` (0.36%) have extremely few training examples, making it inherently difficult for any classifier to learn meaningful patterns for these classes without

Table 2.4: Distribution of the target variable `country_destination` in the training set ($n = 213{,}451$).

Rank	Destination	Percentage (%)
1	NDF	58.35
2	US	29.22
3	other	4.73
4	FR	2.35
5	IT	1.33
6	GB	1.09
7	ES	1.05
8	CA	0.67
9	DE	0.50
10	NL	0.36
11	AU	0.25
12	PT	0.10

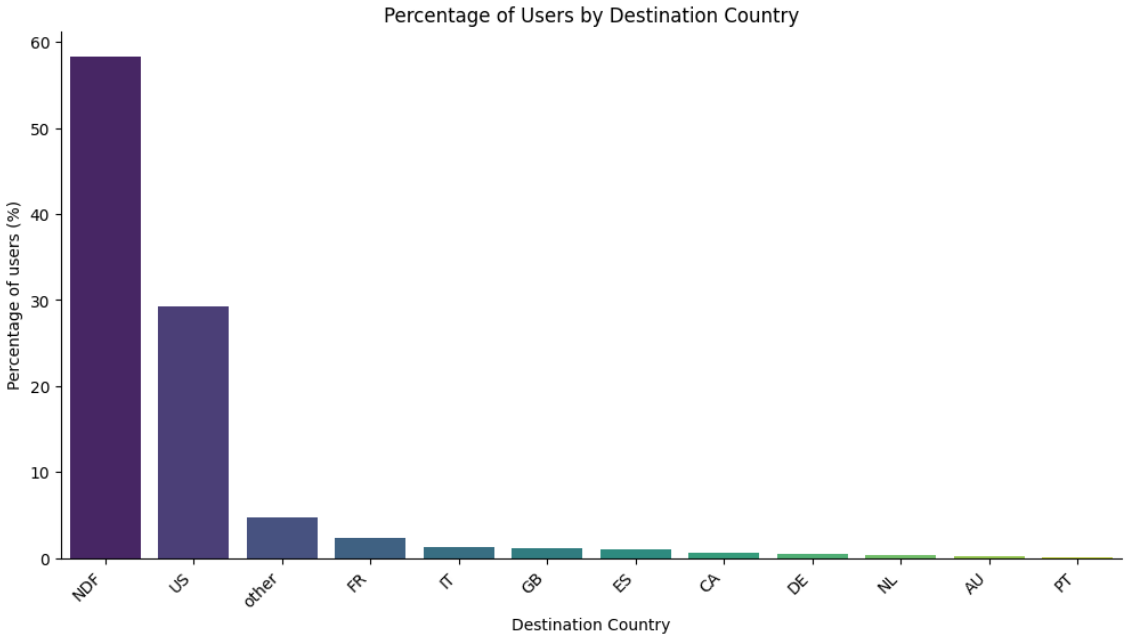

Figure 2.1: Percentage of users by destination country. The label `NDF` (no booking found) dominates at 58.35%, followed by `US` at 29.22%. The remaining 10 destination classes together account for just 12.43% of the training data, illustrating the severe class imbalance present in the dataset.

explicit imbalance-handling strategies (see Chapter 9).

These distributional properties directly motivate several design decisions made throughout this project, including the hierarchical modelling strategy (Chapter 8), the use of SMOTE-based oversampling (Chapter 9), and the choice of NDCG@5 as the primary evaluation metric (Chapter 7).

Chapter 3

Exploratory Data Analysis

Before building any predictive model, a thorough exploratory data analysis (EDA) was conducted to understand the structure, distribution, and relationships present in both the users table and the sessions log. This chapter presents the key findings organised by theme: user demographics, booking destination patterns, signup behaviour, temporal trends, and session activity. All visualisations were generated directly from the merged dataset after the initial data loading step described in Chapter 2.

3.1 User Demographics

3.1.1 Age Distribution

The `age` field is one of the most informative demographic features in the dataset, yet it is also one of the most problematic. Among the 275,547 users in the combined train and test sets, only **158,681** (approximately 57.6%) have a non-null age value. Furthermore, a preliminary inspection of the raw values reveals significant data quality issues:

- **Year entries instead of age** — 828 users entered their birth year (values > 1000, ranging from 1920 to 2014) rather than their actual age. These were corrected by computing $2015 - \text{year}$.
- **Implausibly young ages** — 188 users reported ages below 18, with values as low as 1. According to Airbnb's terms of service, users must be at least 18 years old.
- **Implausibly old ages** — After year correction, the maximum age reached 150, which is biologically impossible. All ages above 90 were capped at 90.

After cleaning, the age distribution has a mean of approximately 36.7 years and a median of 33 years. As shown in Figure 3.1, the distribution is right-skewed with the most common ages concentrated between **25 and 40 years**, reflecting Airbnb's predominantly young-adult user base during this period.

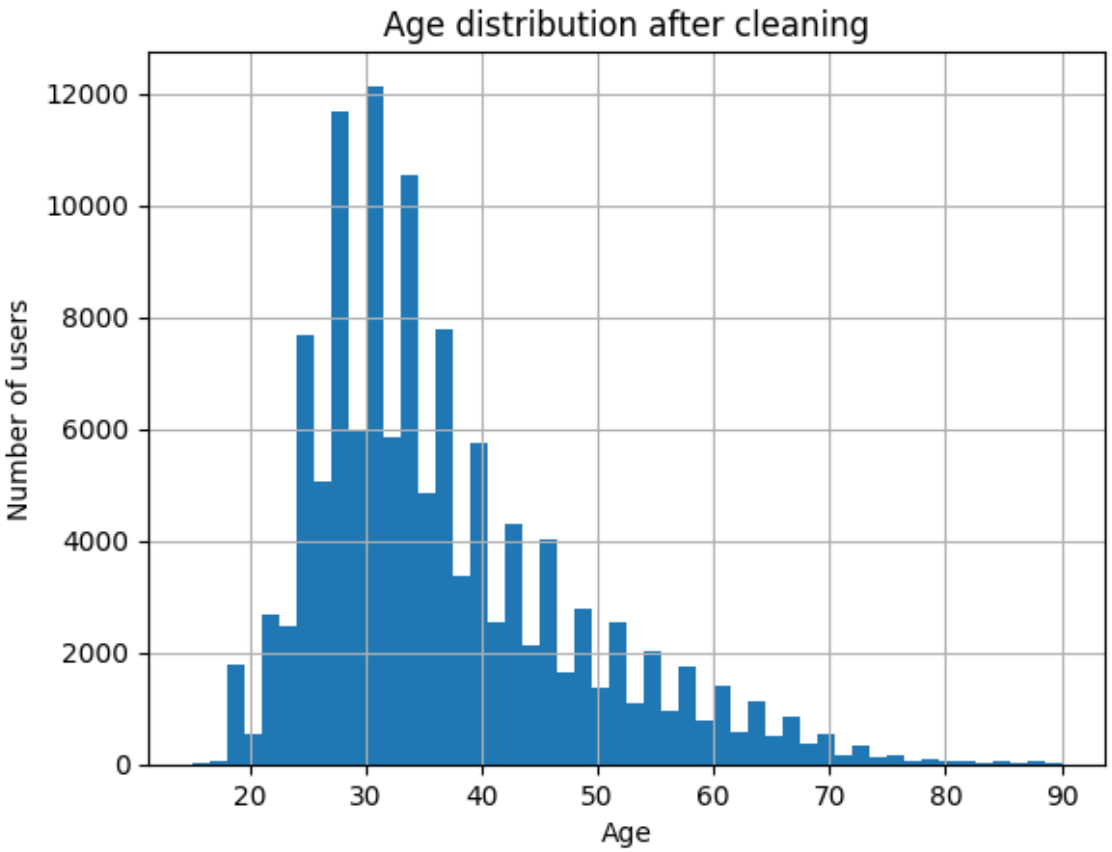

Figure 3.1: Age distribution of Airbnb users after cleaning (kernel density estimate with rug plot). The distribution peaks between ages 25 and 40, with a long right tail indicating a smaller proportion of older users. Capping was applied at age 90.

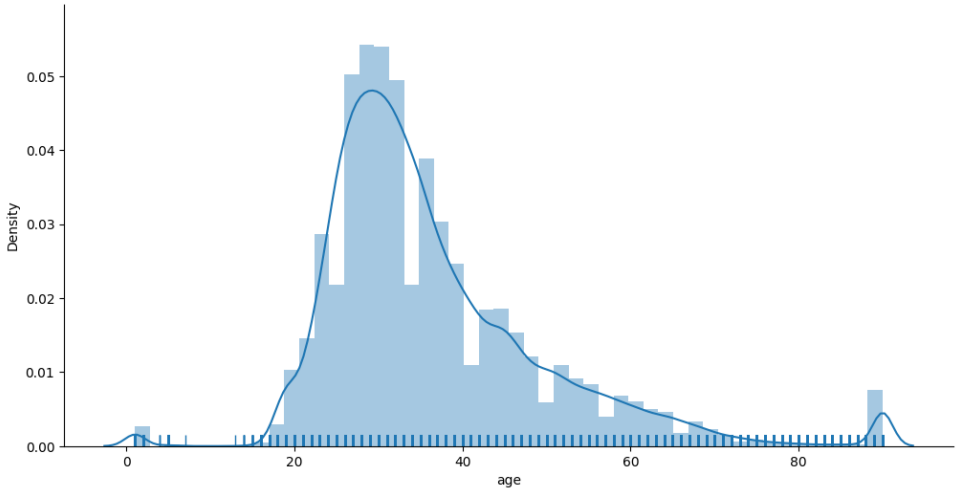

Figure 3.2: Histogram of user age (bins of width 1, range 15–90) after all cleaning steps. The spike at age 33 reflects the median imputation applied to missing values.

3.1.2 Gender Distribution

The `gender` feature takes four values: `MALE`, `FEMALE`, `OTHER`, and `-unknown-`. As shown in Figure 3.3, approximately **45% of users did not disclose their gender**

(labelled `-unknown-`). Among those who did, there is no significant imbalance between male and female users — both groups are represented in roughly equal proportions, suggesting that Airbnb attracts a balanced user base by gender.

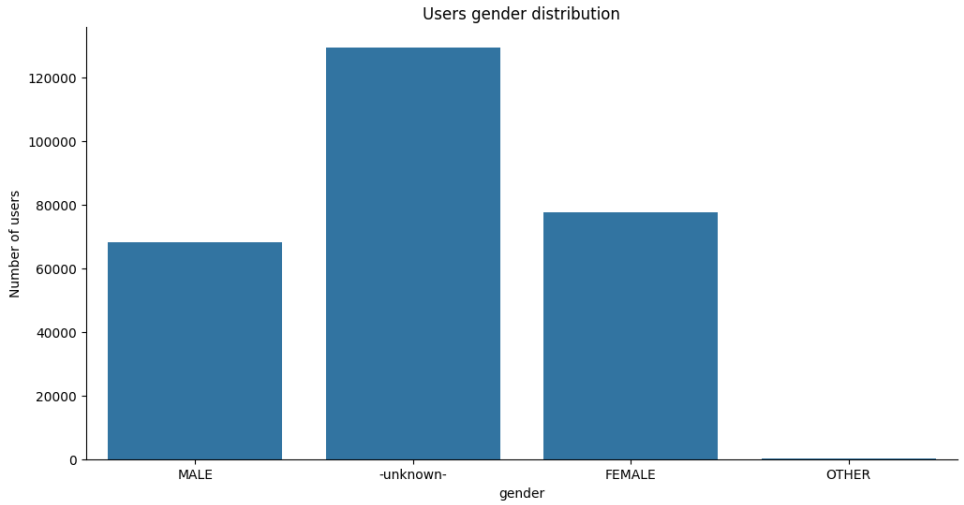

Figure 3.3: Gender distribution across all users. Approximately 45% of users have an unknown gender. Among disclosed genders, the male–female split is nearly even.

3.1.3 Preferred Language

The `language` feature records the user's preferred interface language. Figure 3.4 shows the distribution among users who made at least one booking (excluding `NDF`). English (`en`) is by far the dominant language, as expected given that the dataset covers a largely US-based user population. Notably, when users who booked within the US are excluded, **Chinese (`zh`) emerges as the second most common language**, followed by French (`fr`) and Spanish (`es`), suggesting that international travellers to non-US destinations are more linguistically diverse.

3.2 Booking Destination Analysis

3.2.1 Overall Destination Breakdown

The target variable `country_destination` is heavily skewed, as discussed in Section 3.1 of Chapter 2. Figure 3.6 reinforces this picture with raw counts: the absolute dominance of `NDF` and `US` is immediately apparent, dwarfing all other classes combined.

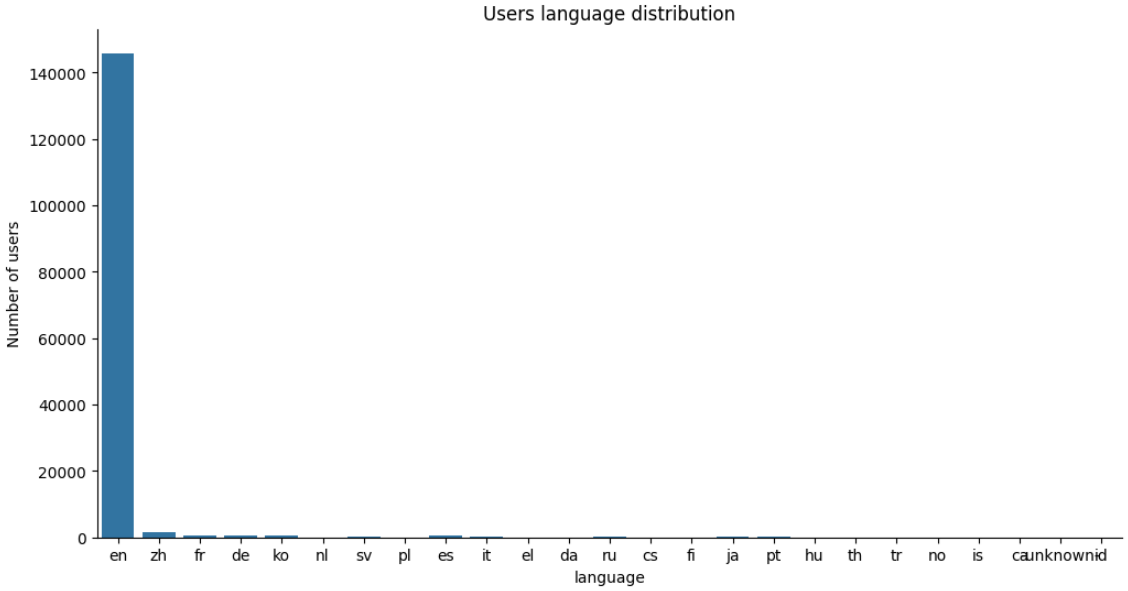

Figure 3.4: Preferred language distribution among bookers (excluding NDF users). English dominates across all destination countries.

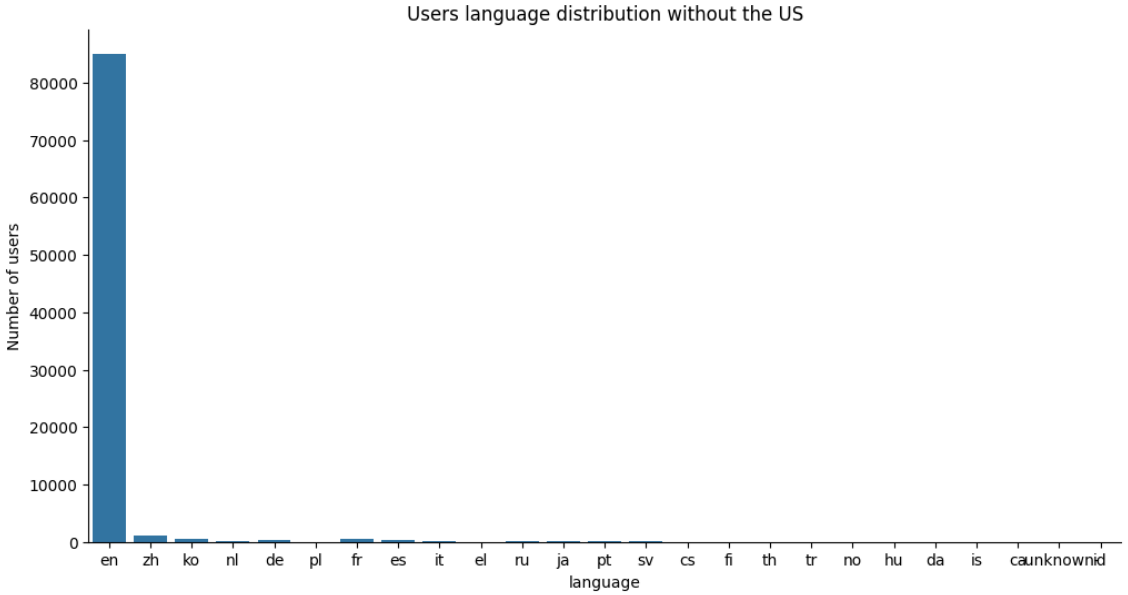

Figure 3.5: Preferred language distribution excluding NDF and US bookers. Chinese becomes the second most common language, highlighting the international nature of non-US travel on the platform.

3.2.2 First Device Type by Destination

Figure 3.7 shows the breakdown of `first_device_type` across booking destinations. Among all bookers, **approximately 60% used an Apple device** (Mac Desktop or iPhone) for their first session, with this share being particularly pronounced among US-bound travellers. However, when US bookings are excluded (Figure 3.8), **Windows Desktop becomes comparably prevalent**, especially in European

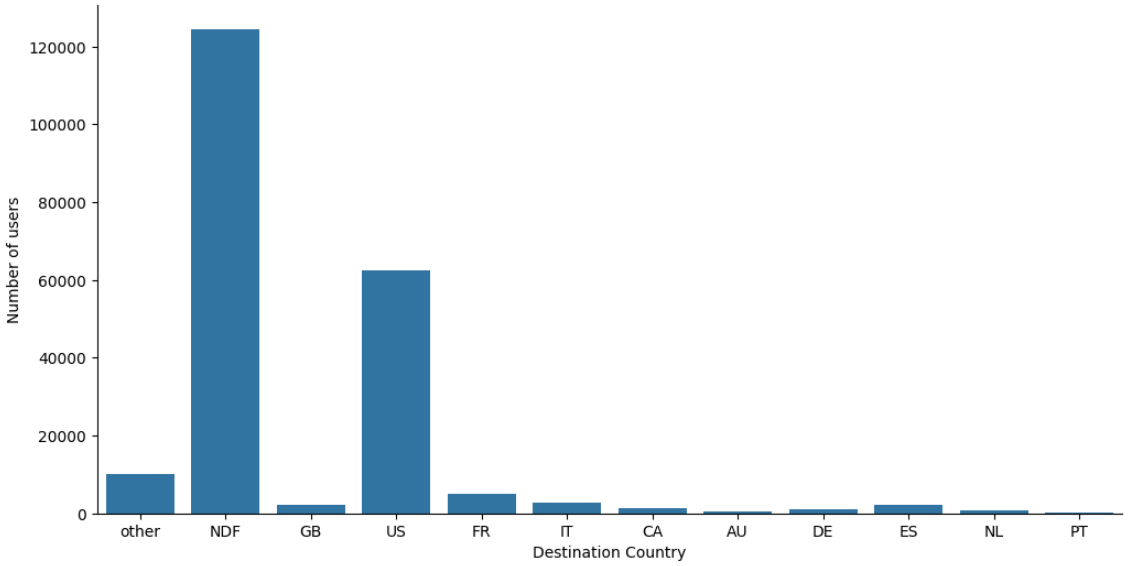

Figure 3.6: Raw user counts by destination country. The scale difference between `NDF`/`US` and all other classes visually illustrates the severity of class imbalance.

destinations such as France, Germany, and Spain. Canada and Australia show the most balanced device split between Mac and Windows desktops.

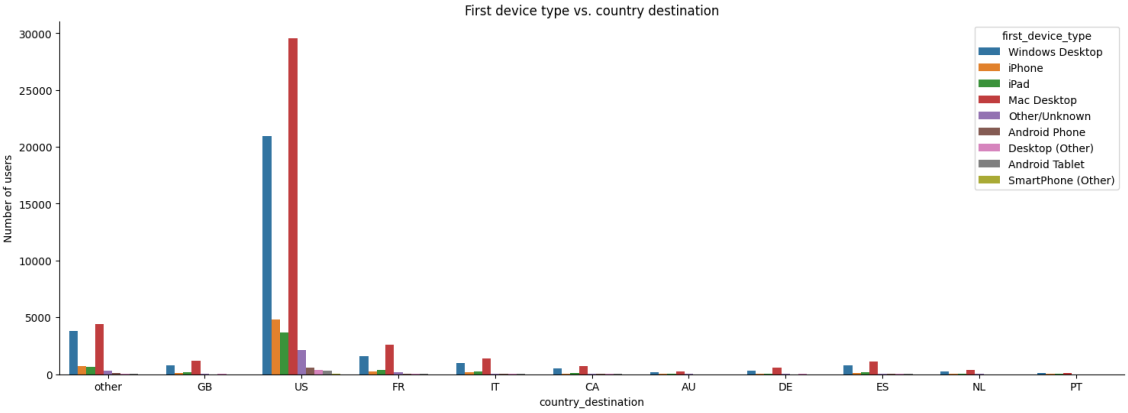

Figure 3.7: First device type by booking destination (excluding NDF). Apple devices dominate, especially for US-bound bookings.

3.3 Signup Behaviour

3.3.1 Signup Method

The `signup_method` feature captures how a user created their Airbnb account. As shown in Figure 3.9, the overwhelming majority of users — **over 70%** — registered via the basic email method. Facebook-based registration accounts for roughly 28% of signups, while Google authentication is used by fewer than 0.3% of users.

When broken down by destination country (Figure 3.10), the basic email method re-

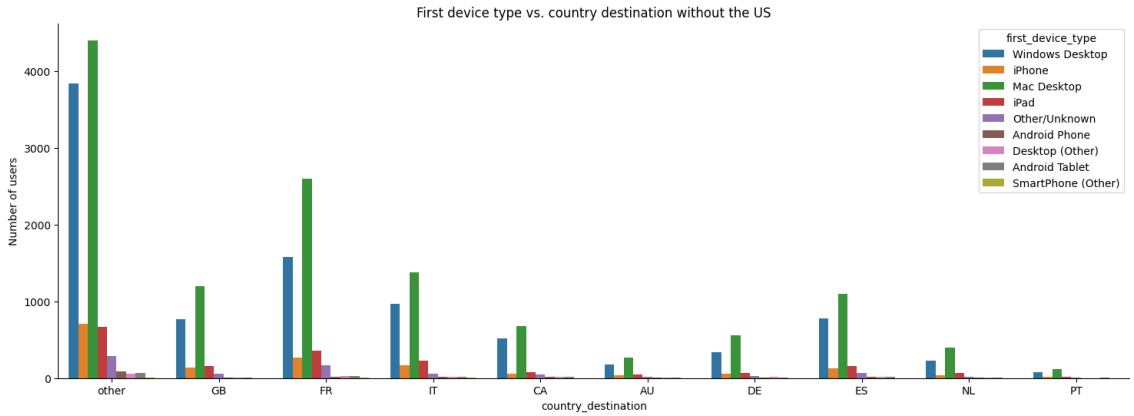

Figure 3.8: First device type by booking destination, excluding NDF and US. Windows Desktop becomes far more prominent for European and Oceanian destinations.

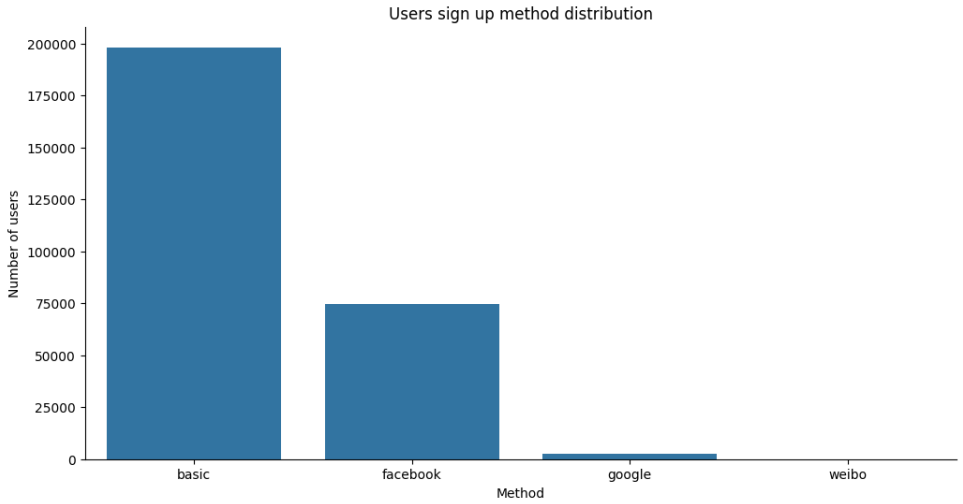

Figure 3.9: Distribution of signup methods across all users. Basic email registration is the dominant method, followed by Facebook.

mains dominant regardless of where the user books. This suggests that `signup_method` alone is not a strong discriminator between destination classes, though it may still contribute marginal signal in conjunction with other features.

3.3.2 Signup Application

The `signup_app` feature records whether the user registered via the web browser (`Web`), iOS app (`iOS`), Android app (`Android`), or mobile web (`Moweb`). As shown in Figure 3.11, **over 85% of all users registered through the Airbnb website**, with iOS being the second most common platform at approximately 10%. Android and Moweb registration together account for the remaining share.

Figure 3.12 confirms that website registration leads across all non-US destination countries. However, iOS adoption shows interesting variation — it is proportionally

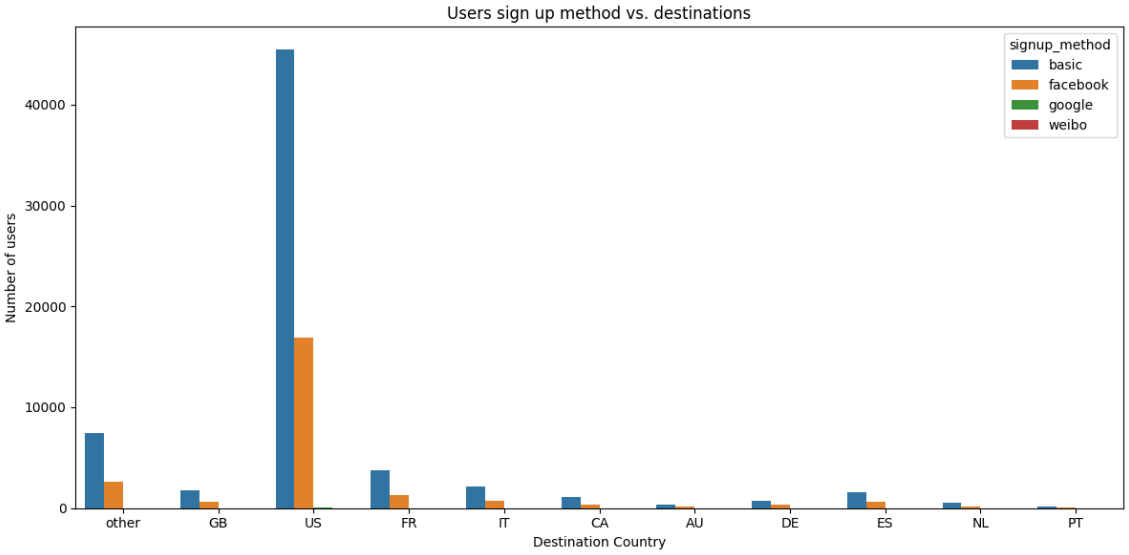

Figure 3.10: Signup method breakdown by booking destination (excluding NDF). Basic email registration leads in all countries.

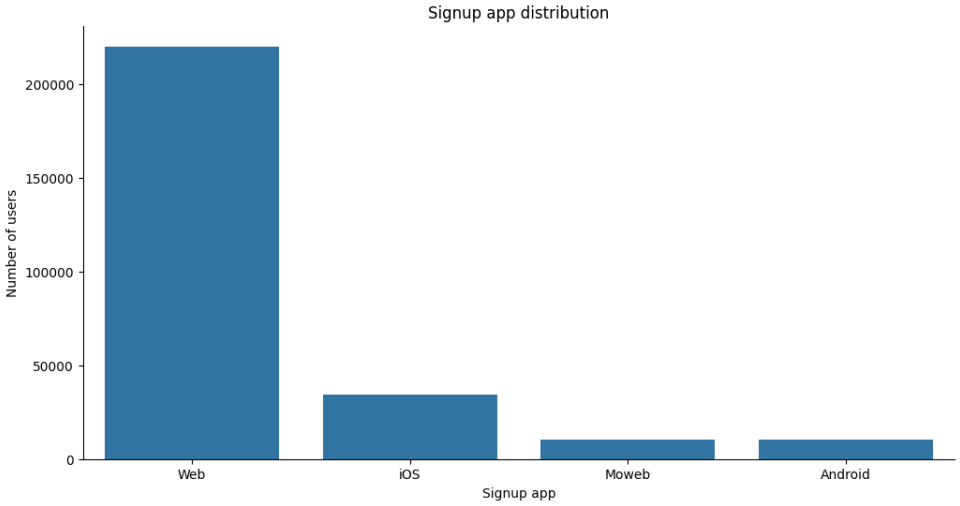

Figure 3.11: Distribution of signup applications. The web platform dominates across all users, followed by iOS.

lower for users, perhaps reflecting lower smartphone penetration in those markets at the time of data collection.

3.4 Temporal Patterns

3.4.1 Account Creation and First Activity Over Time

Two date-based signals are available in the users table: `date_account_created` and `timestamp_first_active`. Figures 3.13 and 3.14 plot the number of events over time for each signal respectively.

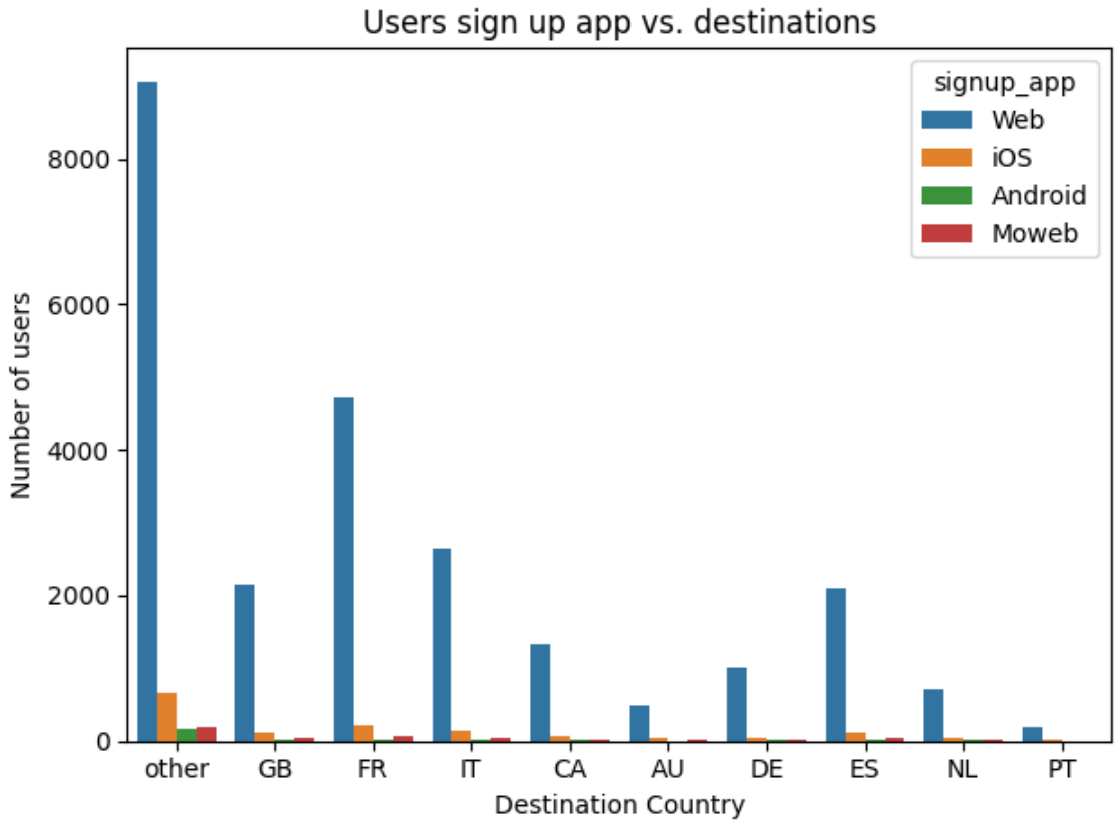

Figure 3.12: Signup application breakdown by destination country (excluding NDF and US). Website registration dominates, though iOS shows slightly higher representation in certain destinations.

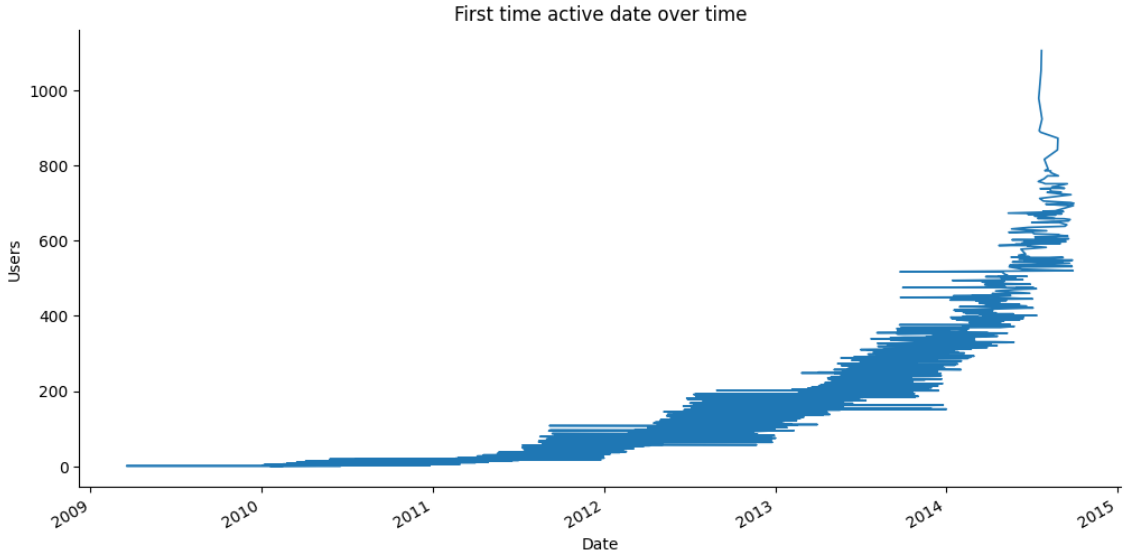

Figure 3.13: Number of users whose first activity falls on each date over time. A strong upward trend is visible from 2010 onwards, with a sharp acceleration after mid-2013.

Both plots tell the same story: **Airbnb grew dramatically between 2010 and 2014**, with a particularly steep acceleration visible from mid-2013 onwards. The near-

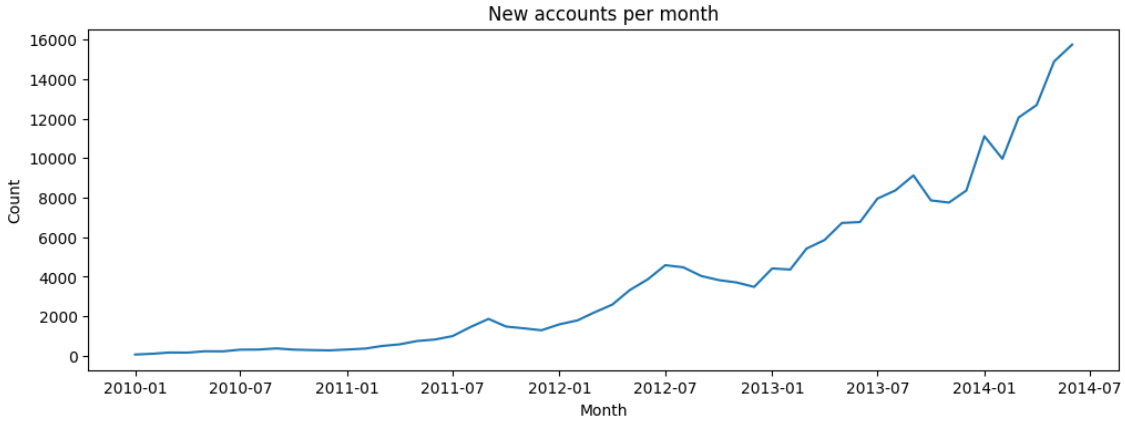

Figure 3.14: Number of new Airbnb accounts created per day over time. The pattern mirrors the first-activity trend closely, confirming consistency between the two date fields and reflecting Airbnb's rapid growth through 2013–2014.

identical shapes of the two curves validate the internal consistency of the date columns. The dataset spans from **2010-01-01** to **2014-06-30** for `timestamp_first_active`, and a similar range for `date_account_created`.

3.4.2 Datetime-Derived Features

To capture temporal patterns that a simple date field cannot express, the following features were extracted from the date columns:

- **Year, month, and weekday** of account creation and first activity — capturing seasonal effects and day-of-week behaviour.
- **Is business day** — a binary flag indicating whether the account creation date falls on a US working day, computed using the `pandas CustomBusinessDay` offset with the US Federal Holiday Calendar.
- **Is holiday** — a binary flag indicating whether the date coincides with a US Federal Holiday, derived from the `USFederalHolidayCalendar`.

These engineered temporal features allow the model to detect, for example, whether users who register on public holidays or weekends exhibit different booking behaviour from those who register on weekdays.

3.5 Session Activity Overview

3.5.1 Coverage and Scale

The sessions table is the largest of the three data files, containing **10,567,737 action records** linked to **135,483 unique users**. Crucially, this represents only approximately **49% of the total user population** (275,547 users across train and test), meaning that just under half of all users have any session activity recorded at all. Users without session records are assigned zero-valued or `"missing"` placeholders during the merging step described in Chapter 6.

3.5.2 Action Types and Volume

The sessions table records six fields per action event. The `action` column contains the specific page or function accessed (e.g., `search_results`, `lookup`, `show`), while `action_type` groups these into higher-level categories: `click`, `view`, `data`, `submit`, `message_post`, `booking_request`, and `booking_response`.

Table 3.1 lists the most frequent actions among rows where `action_type` was missing and required imputation — these predominantly correspond to background data-fetch events (`similar_listings_v2`, `lookup`, `track_page_view`) rather than explicit user clicks.

Table 3.1: Top actions among rows with missing `action_type` values, prior to imputation.

Action	Count
`show`	580,485
`similar_listings_v2`	168,457
`lookup`	161,422
`campaigns`	104,331
`track_page_view`	80,949
`index`	16,682
`localization_settings`	5,380
`uptodate`	3,329

3.5.3 Device Usage in Sessions

The `device_type` column in the sessions table captures the device used for each individual action event, and is distinct from `first_device_type` in the users table (which records only the device used at first activity). The most common device types recorded in sessions are Mac Desktop, Windows Desktop, iPhone, and iPad. These signals are aggregated per user during feature engineering (Chapter 5) into binary flags

(`apple_device`, `desktop_device`, `tablet_device`, `mobile_device`) that describe the user's predominant device category across their entire session history.

3.5.4 Session Duration Patterns

The `secs_elapsed` field measures the time a user spent on each action. Its distribution is highly right-skewed, with most actions lasting only a few seconds but a long tail of very extended interactions. For this reason, aggregate statistics such as **median**, **sum**, **mean**, **min**, and **max** of `secs_elapsed` were computed per user. Additionally, three pause-based features were defined:

- **`day_pauses`** — number of actions where `secs_elapsed` $>$ 86,400 seconds (more than one day).
- **`long_pauses`** — number of actions where `secs_elapsed` $>$ 300,000 seconds (more than 3.5 days).
- **`short_pauses`** — all remaining non-day-pause actions, capturing typical within-session browsing behaviour.

These features encode information about how engaged and persistent a user is on the platform — a user with many day-pauses may be deliberating over a booking, while a user with only short pauses may be casually browsing.

3.5.5 Summary of EDA Findings

The following key insights from EDA directly informed modelling decisions made throughout this project:

1. **Severe class imbalance** in the target variable, with NDF and US together accounting for over 87% of labelled training records, necessitates explicit imbalance-handling strategies.
2. **Age** is the strongest demographic predictor, but requires careful cleaning due to widespread data entry errors.
3. **Gender** has a large unknown proportion ($\sim$45%) but is one of the top features by permutation importance after modelling.
4. **Device type** shows meaningful variation across destination countries, with Apple devices dominating US bookings and Windows desktops being more prevalent in Europe.

Chapter 4

Data Cleaning and Preprocessing

Raw data is rarely ready for modelling. This chapter documents every cleaning and preprocessing step applied to both the sessions table and the users table, in the exact order they were performed in the pipeline. The goal of this stage is to produce a fully populated, type-consistent, and model-ready dataframe with no residual null values before feature engineering begins. All steps described here were implemented in Python using `pandas` and `numpy`.

4.1 Handling Missing Values in Sessions Data

The sessions table was processed first, before any merging with the users table. The raw sessions file contained significant missing data across multiple columns, as summarised in Table 4.1.

Table 4.1: Missing value counts in the raw sessions table (10,567,737 rows).

Column	Missing Values
`user_id`	34,496
`action`	79,626
`action_type`	1,126,204
`action_detail`	1,126,204
`device_type`	0
`secs_elapsed`	136,031

The missing values were addressed in the following sequence:

4.1.1 Step 1 — Drop Rows with Null `user_id`

Rows where `user_id` is null cannot be linked to any user and are therefore meaningless for aggregation. All 34,496 such rows were dropped, reducing the table to **10,533,241 rows**.

```
sessions = sessions[sessions.user_id.notnull()]
```

4.1.2 Step 2 — Impute Missing `action`

After dropping null `user_id` rows, 79,480 rows still had a null `action` value. Inspection revealed that *all* of these rows had `action_type` and `action_detail` equal to `"message_post"` — a background event type that does not correspond to a named page action. All null `action` values were therefore filled with the string `"message"`:

```
sessions.loc[sessions.action.isnull(), 'action'] = 'message'
```

4.1.3 Step 3 — Impute Missing `action_type` and `action_detail`

After Step 2, 1,122,957 rows still had null `action_type` and `action_detail` values. A two-pass strategy was applied:

Pass 1 — Mode imputation by action. A helper function `most_common_value_by_all_users` was defined to compute the most frequent `action_type` for each unique `action` value across all users, then merge this modal value back to fill the nulls. The same function was applied to `action_detail`. This reduced the remaining nulls from 1,122,957 to **415,562**.

Pass 2 — Rule-based imputation for remaining actions. The residual 415,562 nulls were concentrated in a small number of specific action values, as shown in Table 4.2. Each was handled with a targeted rule:

Table 4.2: Remaining null `action_type` values after Pass 1, with counts and imputation strategy applied.

Action	Count	Strategy
`similar_listings_v2`	168,457	Set to `data` / `similar_listings` (inferred from `similar_listings`)
`lookup`	161,422	Set to `lookup` / `lookup`
`track_page_view`	80,949	Set to `track_page_view` / `track_page_view`
All remaining	4,734	Filled with `"missing"`

For `similar_listings_v2`, the existing `similar_listings` action was used as a reference: it consistently had `action_type = "data"` and `action_detail = "similar_listings"` across 363,423 rows. The same values were applied to `similar_listings_v2`. The remaining rare actions were filled with the literal string `"missing"`.

4.1.4 Step 4 — Impute Missing `secs_elapsed`

After resolving the `action`-related nulls, 135,483 rows remained with a null `secs_elapsed`. Rather than using a global constant, the imputation was performed using the **per-action median**:

```
med = sessions.groupby('action')['secs_elapsed'].transform('median')
sessions['secs_elapsed'] = sessions['secs_elapsed'].fillna(med)
```

This approach preserves the characteristic duration associated with each action type (e.g., a `search_results` click tends to last longer than a `lookup` event), producing more realistic imputations than a global mean or zero-fill would. After this step, **all six columns in the sessions table were fully populated with zero null values**.

4.2 Handling Missing Values in Users Data

After the sessions aggregation step (described in Chapter 5), the aggregated `sessions_new` dataframe was merged with the users table. Because the sessions log covers only **135,483 of the 275,547** total users, approximately 51% of users in the merged dataframe had no session record and therefore received null values across all 22 session-derived columns.

Two imputation strategies were applied depending on the column type:

- **Categorical session columns** (`action`, `action_detail`, `action_type`, `device_type`) — filled with the string `"missing"`, which is treated as a valid categorical level by the downstream encoder. This avoids conflating absence of data with any observed category.
- **Continuous session columns** (`action_count`, `apple_device`, `desktop_device`, `mobile_device`, `tablet_device`, `secs_elapsed_max`, `secs_elapsed_mean`, `secs_elapsed_me` `secs_elapsed_min`, `secs_elapsed_sum`, `session_length`, `short_pauses`, `long_pauses`, `day_pauses`, `unique_action_details`, `unique_action_types`, `unique_actions`, `unique_device_types`) — filled with **0**, reflecting the interpretation that a user with no session record performed zero actions on the platform.

After applying both strategies, the remaining null counts across the merged dataframe were reduced to only four columns, as shown in Table 4.3.

Table 4.3: Remaining null values in the merged dataframe after session-level imputation.

Column	Missing Values
`key_0`	140,064
`age`	116,866
`first_affiliate_tracked`	6,085
`country_destination`	62,096

The `key_0` column was a merge artefact and was subsequently dropped. The

`country_destination` nulls correspond entirely to the test set users, which by design have no labels. The remaining two columns — `age` and `first_affiliate_tracked` — are addressed in the sections that follow.

4.3 Age Cleaning

The `age` column required the most intensive cleaning of any feature in the dataset. Before any processing, the raw statistics for non-null age values were as follows:

Table 4.4: Descriptive statistics for `age` before and after cleaning (n = 158,681 non-null users).

Statistic	Before Cleaning	After Cleaning
Mean	47.15	36.71
Std	142.63	14.05
Min	1.00	1.00
25%	28.00	28.00
Median	33.00	33.00
75%	42.00	42.00
Max	2014.00	90.00

Three categories of erroneous entries were identified and corrected:

(a) Year entries instead of age. A total of 828 users entered their birth year (values > 1000, ranging from 1920 to 2014) rather than their current age. These were corrected by computing age = 2015 − year_entered:

```
df_with_year = df['age'] > 1000
df.loc[df_with_year, 'age'] = 2015 - df.loc[df_with_year, 'age']
```

(b) Ages below 18. After the year correction, 188 users still had ages below 18, with a minimum of 1. Although Airbnb's terms of service require users to be at least 18 years old, enforcement is not strict. Ages below 18 were retained in the data but noted as a source of noise.

(c) Ages above 90. After year correction, the maximum age reached 150 — biologically implausible. All ages above 90 were capped at 90:

```
df.loc[df.age > 90, 'age'] = 90
```

(d) Remaining null ages — median imputation. After cleaning, 116,866 age values (approximately 42.4% of all users) remained null. These were imputed with the **median age of 33**, which was computed from the cleaned non-null population. This was done *after* first creating the `age_group` feature (see Chapter 5), so that the group assignment used the original null indicator rather than the imputed value:

```
df.age = df.age.fillna(33)
```

4.4 Date Feature Parsing

The two date columns in the users table — `date_account_created` and `timestamp_first_active` — required type conversion before any temporal features could be extracted from them.

4.4.1 Type Conversion

`date_account_created` was stored as a string in `YYYY-MM-DD` format and was converted using `pd.to_datetime`:

```
df['date_account_created'] = pd.to_datetime(df['date_account_created'])
```

`timestamp_first_active` was stored as a 14-digit integer in the format `YYYYMMDDHHmmss`. Only the date portion was needed, so the timestamp was first integer-divided by 10^6 to strip the time component, then converted:

```
df['timestamp_first_active'] = pd.to_datetime(
    df.timestamp_first_active // 1000000, format='%Y%m%d'
)
```

After conversion, the date ranges in the dataset were confirmed as:

- `timestamp_first_active`: **2009-03-19** to **2014-09-30**
- `date_account_created`: **2010-01-01** to **2014-09-30**

4.4.2 US Holiday and Business Day Calendars

To support the extraction of business-day and holiday flags, the `USFederalHolidayCalendar` from `pandas.tseries.holiday` was instantiated. Holidays and business days were computed over the full date range covered by `timestamp_first_active`.

4.4.3 Derived Temporal Columns

Six new columns were derived from each of the two date fields, yielding **twelve temporal features** in total. Table 4.5 lists all derived columns and their definitions.

Table 4.5: Temporal features derived from `date_account_created` and `timestamp_first_active`.

Feature	Type	Definition
`year_account_created`	Integer	Calendar year of account creation
`month_account_created`	Integer (1–12)	Month of account creation
`weekday_account_created`	Integer (0–6)	Day of week of account creation (0 = Monday)
`business_day_account_created`	Binary (0/1)	1 if account was created on a US business day
`holiday_account_created`	Binary (0/1)	1 if account was created on a US federal holiday
`year_first_active`	Integer	Calendar year of first platform activity
`month_first_active`	Integer (1–12)	Month of first platform activity
`weekday_first_active`	Integer (0–6)	Day of week of first platform activity
`business_day_first_active`	Binary (0/1)	1 if first activity fell on a US business day
`holiday_first_active`	Binary (0/1)	1 if first activity fell on a US federal holiday

4.5 Other Categorical Fixes

Several categorical features in the users table contained either missing values or an excessively large number of low-frequency levels that would inflate the dimensionality after one-hot encoding. A frequency-based grouping strategy was applied uniformly, using a **cut-off threshold of 2,000 occurrences**: any category value appearing fewer than 2,000 times across the combined train and test set was grouped into an `"Other"` bucket. Language used a lower cut-off of 200 given its higher predictive importance.

4.5.1 `first_affiliate_tracked` — Missing Value Imputation

The `first_affiliate_tracked` column had 6,085 null values. Inspection of its value distribution (Table 4.6) showed that `"untracked"` was already the dominant category, representing users whose first visit was not linked to any marketing touchpoint. Null values were semantically equivalent to this category and were therefore imputed accordingly:

```
df.first_affiliate_tracked = df.first_affiliate_tracked.fillna("untracked")
```

After this imputation, only the test set `country_destination` nulls (62,096) remained in the dataframe — all other columns were fully populated.

Table 4.6: Value counts for `first_affiliate_tracked` before null imputation.

Value	Count
`untracked`	143,181
`linked`	62,064
`omg`	54,859
`tracked-other`	6,655
`product`	2,353
`marketing`	281
`local ops`	69
(null)	6,085

4.5.2 `first_browser` — Mobile Consolidation and Low-Frequency Grouping

The `first_browser` column originally contained many fine-grained browser labels. Five mobile browser variants were first consolidated into a single `"Mobile"` category.

Any remaining browser with fewer than 2,000 occurrences was then grouped into `"Other"`. The final distribution after grouping is shown in Table 4.7.

Table 4.7: Final `first_browser` distribution after mobile consolidation and low-frequency grouping.

Browser	Count
`Chrome`	78,671
`Safari`	53,302
`-unknown-`	44,394
`Firefox`	38,665
`Mobile`	34,581
`IE`	24,744
`Other`	1,190

4.5.3 `language` — Low-Frequency Grouping

The `language` column contained 26 distinct language codes. Given the potentially high predictive value of language (a non-English speaker may be more likely to book in their home country), a lower cut-off of **200 occurrences** was used. Languages appearing fewer than 200 times were grouped into `"Other"`. The resulting distribution retained the 10 most common languages individually, as shown in Table 4.8.

A new binary feature `not_English` was also created to flag non-English speakers, providing the model with a simple signal without requiring it to learn the effect from the one-hot encoded language dummies alone:

```
df['not_English'] = df.language.map(lambda x: 0 if x == 'en' else 1)
```

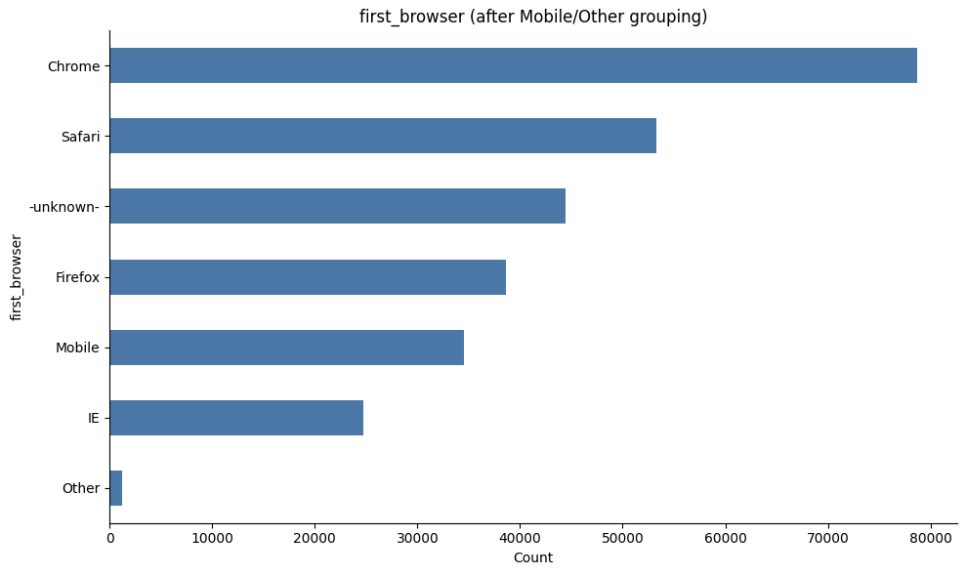

Figure 4.1: Horizontal bar chart of `first_browser` after mobile consolidation and low-frequency grouping. Chrome and Safari are the two dominant browsers, together accounting for nearly half of all users.

Table 4.8: Final `language` distribution after low-frequency grouping (cut-off: 200 occurrences).

Language	Count
en	265,538
zh	2,634
fr	1,508
es	1,174
ko	1,116
de	977
Other	792
it	633
ru	508
ja	345
pt	322

4.5.4 Session Categorical Features — Low-Frequency Grouping

The same cut-off of 2,000 occurrences was applied to four session-derived categorical columns that were carried forward into the merged dataframe. Table 4.9 summarises the before-and-after state for each.

4.5.5 `affiliate_provider` — Low-Frequency Grouping

The `affiliate_provider` column contained 18 distinct values, of which the six most frequent (direct, google, other, facebook, bing, craigslist) all exceeded the 2,000

Table 4.9: Session categorical features before and after low-frequency grouping (cut-off: 2,000). The `missing` value is retained as a separate category.

Feature	Distinct values (before)	Distinct values (after)	Top retained values
action	170	14	missing, show, search_results, index, dashboard, ...
action_detail	96	10	missing, -unknown-, p3, view_search_results, ...
action_type	12	7	missing, view, click, -unknown-, data, submit, Other
device_type	14	10	missing, Mac Desktop, Windows Desktop, iPhone, ...

threshold. All remaining providers were merged into the existing `"other"` bucket, yielding six final levels.

4.5.6 Final State After All Preprocessing

Upon completion of all cleaning and categorical-fix steps, the merged dataframe contained **zero null values** in all columns except `country_destination` (test set labels only). All 13 categorical features were confirmed complete prior to one-hot encoding:

`gender, signup_method, language, affiliate_channel, affiliate_provider, first_affiliate_tracked, signup_app, first_device_type, first_browser, action, action_type, action_detail, device_type`

The fully preprocessed dataframe was then passed to the feature engineering pipeline described in Chapter 5.

Chapter 5

Feature Engineering and Aggregation

Feature engineering is the most consequential stage of the modelling pipeline in this project. The raw data provides only six columns in the sessions table and fifteen in the users table. Through careful aggregation and transformation, this chapter describes how those raw fields were expanded into a rich, **23-column per-user sessions summary** and a set of additional user-level features, ultimately yielding a fully engineered feature matrix ready for modelling.

The feature engineering process was conducted in two stages:

1. **Session-level aggregation** — the raw sessions table (10,533,241 rows, one per action) was collapsed to one row per user (135,483 rows), with each column capturing a different statistical summary of that user's browsing behaviour.
2. **User-level feature construction** — applied directly to the merged users dataframe, covering age grouping, datetime decomposition, and categorical frequency reduction.

All session aggregation was performed on the cleaned sessions table described in Section 4.1, before merging with the users table.

5.1 Session-Level Aggregations

The session aggregation pipeline constructs `sessions_new`, a per-user summary dataframe that captures each user's most representative action patterns, timing statistics, and device usage. The process begins with a single foundational count and is progressively extended through the subsections that follow.

5.1.1 Action Count

The first and most fundamental session feature is `action_count`: the total number of action records logged for each user. This directly captures overall platform

engagement — a user who visited only once will have a low count, while a heavily engaged user browsing listings extensively will have a high count.

```
sessions_new = (
    sessions.groupby('user_id').size()
            .reset_index(name='action_count')
            .sort_values('action_count', ascending=False)
            .reset_index(drop=True)
)
```

This produced a base dataframe of **135,483 rows** — one per unique user appearing in the sessions log — with the highest recorded action count being 2,722 for a single user.

5.1.2 Most Frequent Value per Feature

For each of the four categorical session columns (`action`, `action_type`, `action_detail`, `device_type`), a function `most_frequent_value` was applied to compute the **modal value** of that column for each user across all their session records. These modal values represent the user's most characteristic behaviour on each dimension.

For example, a user whose most frequent action is `show` and whose most common device is `iPhone` will have those values assigned to their row in `sessions_new`. Table 5.1 shows the top five users by action count with their modal values.

Table 5.1: Top 5 users by action count with their modal session feature values.

user_id	action_count	action	action_type	action_detail	device_type
mxqbh3ykxl	2,722	complete_status	-unknown-	-unknown-	Mac Desktop
0hjoc5q8nf	2,644	show	view	p3	Tablet
mjbl6rrj52	2,476	show	-unknown-	-unknown-	iPhone
l5lgm3w5pc	2,424	show	view	user_profile	iPhone
wg9413iaux	2,362	show	view	-unknown-	iPhone

5.2 Unique Value Counts

Beyond the most frequent value, the *diversity* of a user's behaviour is also informative. A user who only ever performed one type of action behaves very differently from one who explored many different pages and features. To capture this, a function `unique_features` was applied to compute the **number of distinct values** of each categorical session column per user.

This was applied to produce four diversity features, as described in Table 5.2.

Table 5.2: Unique value count features derived from the sessions table.

Feature	Definition
`unique_actions`	Number of distinct `action` values performed by the user across all sessions
`unique_action_types`	Number of distinct `action_type` categories encountered by the user
`unique_action_details`	Number of distinct `action_detail` values in the user's session history
`unique_device_types`	Number of distinct device types used by the user across sessions

A user with a high `unique_actions` count is likely an engaged browser who explored many different areas of the platform, while a user with `unique_device_types` > 1 accessed Airbnb from multiple devices — both signals that could correlate with booking probability and destination preference.

5.3 Duration Statistics

The `secs_elapsed` column records the time spent on each individual action event. Since a single user may have hundreds or thousands of action records, five standard descriptive statistics and three engineered pause features were computed per user using a single grouped aggregation.

Table 5.3 provides a full description of each of the nine resulting features.

Table 5.3: Duration and pause features derived from `secs_elapsed`.

Feature	Definition
`secs_elapsed_sum`	Total time (seconds) spent across all of the user's action events
`secs_elapsed_mean`	Mean time per action event
`secs_elapsed_min`	Minimum time recorded in any single action event
`secs_elapsed_max`	Maximum time recorded in any single action event
`secs_elapsed_median`	Median time per action event; more robust to extreme outliers than the mean
`day_pauses`	Number of action events where `secs_elapsed` $>$ 86,400 (more than one full day)
`long_pauses`	Number of action events where `secs_elapsed` $>$ 300,000 (more than 3.5 days)
`short_pauses`	Number of action events where `secs_elapsed` $<$ 3,600 (less than one hour); captures typical within-session browsing
`session_length`	Count of non-zero `secs_elapsed` values; approximates the number of meaningful interactions

The pause features are particularly informative. A user with many `day_pauses` likely returned to the platform multiple times over several days, suggesting deliberate search behaviour. A user with mostly `short_pauses` is likely engaged in a single focused session. These behavioural signals are hypothesised to be predictive of whether the user eventually books, and if so, where.

After merging the `secs_elapsed` aggregation into `sessions_new`, all nine duration columns were confirmed to have **zero null values**, as verified by `sessions_new.isnull().sum()`.

5.4 Device Category Features

The modal `device_type` column (from Section 5.1) captures the single most-used device type for each user. To provide richer, overlapping device-category information, four additional **binary indicator features** were engineered by checking whether the user's most frequent device type falls within a predefined category group:

```
apple_device   = ['Mac Desktop', 'iPhone', 'iPdad Tablet', 'iPodtouch']
desktop_device = ['Mac Desktop', 'Windows Desktop', 'Chromebook',
                  'Linux Desktop']
tablet_device  = ['Android App Unknown Phone/Tablet', 'iPad Tablet',
                  'Tablet']
mobile_device  = ['Android Phone', 'iPhone', 'Windows Phone',
                  'Blackberry', 'Opera Phone']

for device in device_types:
    sessions_new[device] = 0
    sessions_new.loc[
        sessions_new.device_type.isin(device_types[device]), device
    ] = 1
```

Table 5.4 describes each of the four resulting binary features. Note that the categories are deliberately **overlapping**: a Mac Desktop user would have both `apple_device = 1` and `desktop_device = 1`, reflecting the fact that a device can belong to multiple meaningful groupings simultaneously.

At the conclusion of the session aggregation pipeline, `sessions_new` contained **135,483 rows** and **23 columns**, with zero missing values. The complete column list is as follows:

```
user_id, action_count, action, action_type, action_detail, device_type,
      unique_actions, unique_action_types, unique_action_details,
```

Table 5.4: Binary device category features and their constituent device types.

Feature	Device types included
`apple_device`	Mac Desktop, iPhone, iPad Tablet, iPodtouch
`desktop_device`	Mac Desktop, Windows Desktop, Chromebook, Linux Desktop
`tablet_device`	Android App Unknown Phone/Tablet, iPad Tablet, Tablet
`mobile_device`	Android Phone, iPhone, Windows Phone, Blackberry, Opera Phone

`unique_device_types,`
`secs_elapsed_sum, secs_elapsed_mean, secs_elapsed_min, secs_elapsed_max,`
`secs_elapsed_median, day_pauses, long_pauses, short_pauses,`
`session_length,`
`apple_device, desktop_device, tablet_device, mobile_device`

5.5 Datetime-Derived Features

After merging the session features with the users table, ten temporal features were constructed from the two date columns `date_account_created` and `timestamp_first_active`. These features capture the *when* of a user's arrival on the platform and allow the model to learn seasonal, day-of-week, and calendar effects that a raw date string cannot express.

Five features were derived from `date_account_created` and five from `timestamp_first_active`, following an identical extraction pattern for each.

The `year` feature captures the strong platform-growth trend visible in Figure 3.14 — users who joined in 2014 behave differently from those who joined in 2010, both in terms of available platform features and in terms of Airbnb's user base composition. The `month` feature allows the model to detect seasonal booking patterns. The `weekday` feature captures day-of-week effects (e.g., users who sign up on a weekend may differ in intent from weekday registrants). The binary `business_day` and `holiday` flags add finer calendar-level context using the US Federal Holiday Calendar.

The original raw date columns were retained in the dataframe during exploration but are not included as model features after decomposition, since their information is fully captured by the derived columns.

5.6 Age Group Feature

Although the cleaned `age` feature is used as a continuous variable after median imputation, an additional **ordinal `age_group` feature** was engineered *before* imputation, so that users with genuinely missing ages could be assigned a dedicated group (0) rather than being incorrectly placed into an age bracket:

```
df['age_group'] = df.age.map(lambda x:
    0 if math.isnan(x) else (    # Group 0: age unknown
    1 if x < 18 else (           # Group 1: under 18
    2 if x <= 32 else (          # Group 2: 18-32 (young adults)
    3 if x <= 42 else 4))))      # Group 3: 33-42 / Group 4: 43+
```

Table 5.5 summarises the five age groups and their interpretation.

Table 5.5: Age group encoding applied to the `age_group` feature.

Code	Age range	Interpretation
0	*NaN (unknown)*	Age was not provided; treated as a separate category rather than imputed before grouping
1	< 18	Underage users (below Airbnb's minimum age)
2	$18 \leq \text{age} \leq 32$	Young adults; the core Airbnb demographic
3	$33 \leq \text{age} \leq 42$	Mid-age adults
4	> 42	Older adults

The `age_group` feature serves as a complementary signal to the continuous `age` variable. It allows tree-based models to split cleanly on age cohorts, and it explicitly encodes the missingness information that would otherwise be lost after the median imputation step. The EDA showed that `age_group` ranked as the **second most important feature** by permutation importance from the baseline Decision Tree (after `gender_-unknown-`), confirming its predictive value.

5.7 Categorical Grouping

Prior to one-hot encoding, all categorical features were reduced in cardinality using the frequency-based grouping strategy described in Section 4.5. The purpose of this step within the feature engineering context is twofold:

1. **Dimensionality control** — one-hot encoding a feature with 170 distinct values (as `action` originally had) would add 170 binary columns to the feature

matrix. Grouping rare values into an `"Other"` bucket reduces this to 14 columns, keeping the feature matrix manageable.

2. **Noise reduction** — categories with very few examples (e.g., fewer than 2,000 occurrences) are unlikely to yield reliable statistical patterns and may cause overfitting in tree-based models.

5.7.1 One-Hot Encoding

After all grouping was applied, the 13 categorical features were one-hot encoded using `pd.get_dummies` with `drop_first=False` (all dummy columns retained):

```
for feature in cat_features:
    dummies = pd.get_dummies(df[feature], prefix=feature,
                             drop_first=False)
    df = df.drop(feature, axis=1)
    df = pd.concat([df, dummies], axis=1)
```

The 13 categorical features that were encoded are listed in Table 5.6, together with their final cardinality after grouping.

Table 5.6: Categorical features one-hot encoded and their final cardinality after low-frequency grouping.

Feature	**Source table**	**Levels**
`gender`	Users	4
`signup_method`	Users	4
`language`	Users	11
`affiliate_channel`	Users	8
`affiliate_provider`	Users	6
`first_affiliate_tracked`	Users	7
`signup_app`	Users	4
`first_device_type`	Users	9
`first_browser`	Users	7
`action`	Sessions	14
`action_type`	Sessions	7
`action_detail`	Sessions	10
`device_type`	Sessions	9
Total OHE columns		**100**

5.7.2 Continuous Feature Scaling

All continuous features were normalised using **Min-Max scaling** to bring each value into the $[0, 1]$ range. This is particularly important for models sensitive to feature

magnitude such as Linear SVM and Logistic Regression, and has no negative effect on tree-based models:

```
min_max_scaler = preprocessing.MinMaxScaler()
for feature in cont_features:
    df[feature] = min_max_scaler.fit_transform(df[[feature]])
```

The continuous features that were scaled include: `age`, `signup_flow`, `action_count`, `unique_actions`, `unique_action_types`, `unique_action_details`, `unique_device_types`, `secs_elapsed_sum`, `secs_elapsed_mean`, `secs_elapsed_min`, `secs_elapsed_max`, `secs_elapsed_median`, `day_pauses`, `long_pauses`, `short_pauses`, `session_length`, and all temporal integer features.

5.7.3 Summary of the Final Feature Matrix

At the conclusion of all feature engineering steps, the combined train-and-test dataframe contains a fully populated feature matrix with **zero null values** (except `country_destination` for test rows). The final feature matrix is composed of:

- **∼100 one-hot encoded dummy columns** from 13 categorical features.
- **9 duration and pause statistics** from `secs_elapsed` aggregations.
- **4 unique-value-count features** from session diversity.
- **4 binary device category flags** from session device type grouping.
- **10 datetime-derived features** from date decomposition.
- **2 age-related features**: continuous `age` (scaled) and ordinal `age_group`.
- **1 language flag**: binary `not_English`.
- **1 action count**: `action_count` (scaled).

This engineered matrix is then split back into training and test sets and passed to the modelling pipeline described in Chapter 7.

Chapter 6

Dataset Merging

With the sessions table fully cleaned and aggregated into a per-user summary (`sessions_new`, 135,483 rows × 23 columns) and the users table preprocessed, the two sources were combined into a single modelling dataframe. This chapter documents the exact merge strategy, explains the motivation behind the design choices made, and describes the final encoding, scaling, and train/test reconstruction steps that produce the feature matrix handed to every model in the pipeline.

The three shapes entering this stage were:

Table 6.1: Shapes of the three dataframes at the start of the merging stage.

Dataframe	Rows	Columns	Description
`sessions_new`	135,483	23	Per-user session aggregates
`train`	213,451	16	Labelled user records
`test`	62,096	15	Unlabelled user records

6.1 Left-Joining Sessions onto Users

6.1.1 Motivation for a Left-Join Strategy

The most natural approach to merging the sessions data with the users table would be a simple inner join — keeping only users who appear in both tables. However, this would silently discard the **140,064 users** (approximately 51% of the combined train and test set) who have no session record at all. These users are not ignorable: they represent real Airbnb registrants and carry their own demographic signals (`age`, `gender`, `signup_method`, etc.) that are still informative for prediction. Dropping them would introduce a severe selection bias, since users without session activity are overwhelmingly more likely to belong to the `NDF` class — removing them would artifically deflate the majority class and distort the training distribution.

The chosen strategy therefore preserves *all* users by performing a **left join**, retaining every user from the users table and filling missing session columns with sentinel values (described in Chapter 4).

6.1.2 Two-Part Merge Pattern

A subtle complication arose because a direct left join via `pd.merge(train, sessions_new, left_on='id', right_on='user_id', how='left')` produces a merged dataframe that contains the `key_0` artefact column from the original Kaggle-provided data. To avoid duplicated index labels and to preserve the correct row ordering for the final split reconstruction, the merge was implemented as a **two-part concatenation pattern** applied independently to both `train` and `test`.

This pattern works as follows. The inner merge (`train1` / `test1`) picks up every user that has a matching entry in `sessions_new` and attaches all 22 session-derived columns to their row. The anti-join (`train2` / `test2`) then captures all users who had *no* session record, preserving them with `NaN` in every session column. The concatenation then stacks both halves, yielding a complete left-join result without dropping a single user.

6.1.3 Post-Merge Column Drops

After the merge, two columns were dropped that were no longer needed:

- **`key_0`** — a merge artefact introduced when the inner join used `train['id']` as the left key. This column was a duplicate of `id` and had 140,064 null values (corresponding to the unmatched users). It was dropped immediately after confirming the merge was successful.
- **`date_first_booking`** — this column records when a user made their first booking and had **186,639 null values** (71% of all rows), making it almost entirely unusable as a predictive feature. More critically, it would constitute *target leakage*: any non-null value in this column implies that the user did make a booking, which directly reveals whether the target class is `NDF` or not.
- **`user_id`** — a duplicate of `id` introduced by the session merge's right key.

```
df.drop(['user_id', 'date_first_booking'], axis=1, inplace=True)
```

6.1.4 Null State After Merging

Immediately after the merge and column drops, and before any imputation, the null counts in the combined dataframe were as shown in Table 6.2. All session-derived columns are null for the 140,064 users who had no session record.

After applying the categorical and continuous session imputation described in Section 4.2, and addressing `age` and `first_affiliate_tracked` as described in Sec-

Table 6.2: Selected null counts in the combined dataframe immediately after merging and before session imputation. All 22 session-derived columns share the same null count of 140,064.

Column (selection)	Null count
`age`	116,866
`first_affiliate_tracked`	6,085
`country_destination`	62,096
`action_count`	140,064
`action`	140,064
`secs_elapsed_sum`	140,064
`apple_device`	140,064
(all other session columns)	140,064

tions 4.3 and 4.5, the only remaining nulls were the 62,096 `country_destination` values belonging to the test split — which by design have no label and are the output to be predicted.

6.1.5 Label Separation

Before encoding, the target column was extracted and stored separately to prevent it from entering the feature matrix:

```
labels = pd.DataFrame(df.country_destination)
df     = df.drop("country_destination", axis=1)
df.drop('id', axis=1, inplace=True)
```

The resulting `labels` dataframe holds 275,547 rows, of which the first 213,451 correspond to training users (with valid string labels) and the remaining 62,096 correspond to test users (all `NaN`). The first five label values confirmed correct ordering:

```
other, NDF, NDF, NDF, GB, ...
```

Figure **??** illustrates the complete merge pipeline from raw input files to the final joint dataframe.

6.2 Encoding and Scaling

With the combined dataframe `df` fully populated and the label column extracted, the final preprocessing steps before modelling were encoding of categorical features and scaling of continuous features.

6.2.1 Feature Partitioning

All columns were partitioned into two lists prior to encoding:

- **Categorical features** (`cat_features`) — 13 columns that hold string labels and require one-hot encoding:

  `gender, signup_method, language, affiliate_channel, affiliate_provider, first_affiliate_tracked, signup_app, first_device_type, first_browser, action, action_type, action_detail, device_type`

- **Continuous features** (`cont_features`) — 20 columns that hold numeric values and require Min-Max scaling:

  `age, signup_flow, action_count, unique_actions, unique_action_types, unique_action_details, unique_device_types, secs_elapsed_sum, secs_elapsed_mean, secs_elapsed_min, secs_elapsed_max, secs_elapsed_median, day_pauses, long_pauses, short_pauses, session_length, apple_device, desktop_device, tablet_device, mobile_device, age_group, not_English, year_account_created, month_account_created, weekday_account_created, ...`

6.2.2 One-Hot Encoding

Each categorical feature was one-hot encoded using `pandas.get_dummies` with `drop_first=False`, meaning all dummy columns were retained without dropping a reference category. This choice was deliberate: tree-based models do not suffer from the multicollinearity that motivates dummy-variable dropping in linear models, and retaining all columns gives the models maximum signal.

```
for feature in cat_features:
    dummies = pd.get_dummies(df[feature], prefix=feature,
                             drop_first=False)
    df = df.drop(feature, axis=1)
    df = pd.concat([df, dummies], axis=1)
```

The encoding was applied to all 13 categorical features in sequence, with each step confirmed by a printed completion message. The 13 features expanded into approximately **100 binary dummy columns** in total, as summarised in Table 6.3.

Table 6.3: One-hot encoding expansion summary for all 13 categorical features.

Feature	Source	Dummy cols
gender	Users	4
signup_method	Users	4
language	Users	11
affiliate_channel	Users	8
affiliate_provider	Users	6
first_affiliate_tracked	Users	7
signup_app	Users	4
first_device_type	Users	9
first_browser	Users	7
action	Sessions	14
action_type	Sessions	7
action_detail	Sessions	10
device_type	Sessions	9
Total		**≈100**

6.2.3 Min-Max Scaling of Continuous Features

All continuous features were normalised to the $[0, 1]$ range using `sklearn.preprocessing.MinMaxScal` applied column by column across the entire combined dataframe (train + test rows together):

```
from sklearn import preprocessing
min_max_scaler = preprocessing.MinMaxScaler()

for feature in cont_features:
    df[feature] = min_max_scaler.fit_transform(df[[feature]])
```

Fitting the scaler on the full combined dataframe (rather than on training rows only) ensures that the scale is consistent between the training and test portions when they are later separated. Since the test labels are not used at any point during this step, there is no information leakage. Min-Max scaling was chosen over standardisation (`StandardScaler`) because the duration and count features have highly right-skewed distributions where a z-score normalisation would leave many values compressed near zero while a small number of high-activity users dominate the upper tail. Mapping to $[0, 1]$ provides a more interpretable and uniformly bounded representation.

6.2.4 Final Feature Matrix Dimensions

After one-hot encoding and scaling, the full combined feature matrix `df` has the following dimensions:

The final shape of the feature matrix was confirmed as **(275,547, 132)** — 132

Table 6.4: Feature matrix dimensions after full encoding and scaling.

Property	**Value**
Total rows (train + test)	275,547
Total feature columns	132
Null values in feature matrix	0
Null values in labels (test)	62,096

features per user, with complete coverage and zero missing values in the predictor columns.

6.3 Train/Test Split Reconstruction

6.3.1 Kaggle Split Reconstruction

Because the train and test sets were stacked into a single dataframe `df` for joint preprocessing, they needed to be separated again before modelling. The original split boundary was preserved by recording the length of the training set (`len(train) = 213,451`) before concatenation. The reconstruction is therefore exact and index-safe:

```
df_train = df[:len(train)]    # rows 0 .. 213450
df_test  = df[len(train):]    # rows 213451 .. 275546

y = labels[:len(train)]       # training labels only (213,451 rows)
```

The test labels remain entirely withheld — the 62,096 `NaN` values in `labels[len(train):]` are never used during training or validation.

6.3.2 Validation Split

For local model evaluation, the training portion was further split into a **training set** (80%) and a **held-out validation set** (20%) using stratified sampling to preserve the class distribution across both halves:

```
from sklearn.model_selection import train_test_split

y_1d = np.ravel(y)   # convert (n,1) DataFrame to 1-D array

X_train, X_val, y_train, y_val = train_test_split(
    X, y_1d,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y_1d
)
```

The resulting split sizes are shown in Table 6.5.

Table 6.5: Validation split sizes after stratified 80/20 partitioning of the training data.

Split	Rows	Columns
X_train	170,760	132
X_val	42,691	132
y_train	170,760	1
y_val	42,691	1

6.3.3 Stratification Rationale

The `stratify=y_1d` argument is essential given the severe class imbalance documented in Chapter 2. Without stratification, random sampling could by chance produce a validation set that over-represents or under-represents the rare minority classes (such as `PT` at 0.10% or `AU` at 0.25%), making validation metrics unreliable. Stratified splitting guarantees that the class proportions in `y_val` match those in the full training set, as confirmed in Table 6.6.

Table 6.6: Class sample counts in the validation set `y_val` (42,691 rows, stratified).

Class	Val count	Val %
NDF	24,909	58.34
US	12,473	29.22
other	2,019	4.73
FR	1,005	2.35
IT	567	1.33
GB	465	1.09
ES	450	1.05
CA	286	0.67
DE	212	0.50
NL	152	0.36
AU	108	0.25
PT	43	0.10
Total	**42,691**	**100.00**

The percentages in Table 6.6 match the full training distribution from Table 2.4 to within rounding error, confirming that the stratified split correctly mirrors the overall class balance. This validation set was then used as the benchmark for comparing all models described in Chapters 7 through 10.

Chapter 7

Modeling

This chapter presents the five classification models trained in Phase 1 of the modelling roadmap, from a simple logistic regression baseline through to GPU-accelerated gradient boosting. Each model is described in terms of its architecture, hyperparameter choices, and performance across six evaluation metrics, with particular emphasis on the competition's primary metric, NDCG@5. A label encoder was applied to convert the 12 string class labels into integers for models that require numeric targets (XGBoost, LightGBM, CatBoost), using `sklearn.preprocessing.LabelEncoder` fitted on the training labels.

All models were trained on `X_train` (170,760 samples $\times$ 132 features) and evaluated on the held-out validation set `X_val` (42,691 samples), constructed via the stratified 80/20 split described in Section 6.3.

7.1 Evaluation Metric: NDCG@5

The Kaggle competition scores submissions using **Normalised Discounted Cumulative Gain at rank 5 (NDCG@5)**. For each user, the model is permitted to submit up to five ranked destination predictions. The metric rewards placing the correct destination country as high in the ranked list as possible, with diminishing credit for correct answers appearing lower in the list.

Formally, for a single user the Discounted Cumulative Gain at rank k is:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)} \tag{7.1}$$

where $\text{rel}_i \in \{0, 1\}$ is 1 if the prediction at rank i is the correct destination and 0 otherwise. Normalisation divides by the ideal DCG (IDCG@$k = 1.0$ when the correct answer is ranked first), giving:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} \tag{7.2}$$

A perfect score of 1.0 means every user's true destination was predicted first. The practical consequence for modelling is that *probability calibration matters*: it is not sufficient to get the correct answer somewhere in the top 5; a model that consistently ranks the true destination first will substantially outperform one that buries it at rank 5.

In addition to NDCG@5, five supplementary metrics were tracked for each model:

- **Accuracy** — proportion of users where the top-1 prediction is correct.
- **Macro F1** — unweighted mean of per-class F1 scores; sensitive to performance on minority classes.
- **Weighted F1** — class-frequency-weighted mean of per-class F1 scores; dominated by NDF and US performance.
- **Macro Recall** — unweighted mean of per-class recall scores.
- **Top-5 Accuracy** — proportion of users where the true destination appears anywhere in the top-5 predicted classes.

NDCG@5 was computed using `sklearn.metrics.ndcg_score` with true labels binarised via `LabelBinarizer` and predicted probabilities (or decision scores for SVM) as the score matrix.

7.2 Logistic Regression (Baseline)

7.2.1 Configuration

A multinomial Logistic Regression was trained as the first baseline to establish a lower bound on performance and to verify that the feature matrix was correctly constructed. The model was configured with balanced class weights to partially counter the NDF/US dominance:

```
from sklearn.linear_model import LogisticRegression

baseline = LogisticRegression(
    max_iter=200,
    n_jobs=-1,
    class_weight="balanced"
)
baseline.fit(X_train, y_train)
```

The solver used was the default `lbfgs` with a maximum of 200 iterations and all CPU cores parallelised via `n_jobs=-1`.

7.2.2 Results

Table 7.1 shows the per-class classification report and Table 7.2 the aggregate metrics. The balanced class weight caused the model to spread predictions across minority classes, resulting in very low precision for rare destinations but higher recall.

Table 7.1: Per-class precision, recall and F1 for Logistic Regression on the validation set.

Class	Precision	Recall	F1	Support
AU	0.00	0.20	0.01	108
CA	0.02	0.04	0.02	286
DE	0.01	0.28	0.01	212
ES	0.02	0.03	0.02	450
FR	0.04	0.15	0.06	1,005
GB	0.03	0.01	0.02	465
IT	0.01	0.04	0.02	567
NDF	0.74	0.49	0.59	24,909
NL	0.00	0.02	0.01	152
PT	0.00	0.21	0.01	43
US	0.45	0.01	0.02	12,475
other	0.07	0.07	0.07	2,019
Macro avg	0.12	0.13	0.07	42,691
Weighted avg	0.53	0.30	0.35	42,691

Table 7.2: Aggregate metrics for Logistic Regression.

Metric	Value
Accuracy	0.2964
Macro F1	0.0707
Weighted F1	0.3532
Macro Recall	0.1287
Top-5 Accuracy	0.5528
NDCG@5	**0.4215**

7.2.3 Permutation Feature Importance

Permutation importance (3 repeats, `random_state=42`) on the validation set revealed that `gender_-unknown-` (0.054) and `age_group` (0.047) are by far the most informative features for the logistic model, followed distantly by `gender_FEMALE` (0.018) and `gender_MALE` (0.014). The model relies almost entirely on demographic signals and essentially ignores session-derived features, suggesting that the linear

decision boundary is insufficient to exploit the non-linear interactions present in the temporal and session data.

The NDCG@5 of **0.4215** establishes the lower bound for all subsequent models.

7.3 Linear SVM

7.3.1 Configuration

A Linear Support Vector Machine (`LinearSVC`) was trained as the second baseline. Unlike kernel SVMs, `LinearSVC` scales efficiently to the 170,760-row training set and naturally exploits the one-hot-encoded sparse feature matrix. Balanced class weights were applied:

```
from sklearn.svm import LinearSVC

svm = LinearSVC(
    class_weight="balanced",
    random_state=42
)
svm.fit(X_train, y_train)
```

Because `LinearSVC` does not produce probability estimates, NDCG@5 was computed using the raw **decision function scores** in place of probabilities:

```
scores = svm.decision_function(X_val)  # shape: (n_samples, n_classes)
ndcg5  = ndcg_score(y_true_bin, scores, k=5)
```

7.3.2 Results

Table 7.3: Aggregate metrics for Linear SVM.

Metric	Value
Accuracy	0.4475
Macro F1	0.0790
Weighted F1	0.4237
Macro Recall	0.1115
Top-5 Accuracy	0.7110
NDCG@5	**0.5858**

The SVM achieves substantially higher accuracy (44.75%) and NDCG@5 (0.5858) than logistic regression, driven primarily by its ability to correctly classify the dominant NDF and US classes without scattering predictions into minority classes

as aggressively. Permutation importance shows that the SVM leans heavily on temporal features: `year_account_created` (0.127), `month_first_active` (0.120), `month_account_created` (0.116), and `year_first_active` (0.106) are the four most important features, contrasting sharply with the demographic dominance seen in logistic regression. This suggests that when and how recently a user joined Airbnb contains strong signal about booking behaviour — a reflection of the platform's rapid growth trajectory during the 2010–2014 period covered by the dataset.

7.4 Decision Tree

7.4.1 Configuration

A single Decision Tree was included to provide an interpretable tree-based baseline and to serve as a point of comparison for the ensemble boosting methods. Depth was capped at 20 and a minimum leaf size of 50 was enforced to prevent severe overfitting:

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    random_state=42,
    class_weight="balanced",
    max_depth=20,
    min_samples_leaf=50
)
dt.fit(X_train, y_train)
```

7.4.2 Results

Table 7.4: Aggregate metrics for Decision Tree.

Metric	Value
Accuracy	0.1514
Macro F1	0.0497
Weighted F1	0.2271
Macro Recall	0.1119
Top-5 Accuracy	0.7472
NDCG@5	**0.4471**

The Decision Tree shows a striking pattern: its top-1 accuracy (15.1%) is the lowest of all five models, yet its Top-5 Accuracy (74.7%) exceeds both logistic regression and the SVM. This occurs because the tree's balanced class weights force it to explore

minority class branches that produce diverse top-5 predictions even when the top-1 prediction is wrong. However, the NDCG@5 of 0.4471 is only marginally better than logistic regression (0.4215), indicating that the diversity of the top-5 set is achieved at the cost of correct rank ordering within it.

Permutation importance highlights `age_group` (0.066), `signup_app_Web` (0.041), `gender_-unknown-` (0.037), and `month_account_created` (0.016) as the tree's primary decision features — a mix of demographic and temporal signals consistent with the EDA findings.

7.5 XGBoost

7.5.1 Configuration

XGBoost (`XGBClassifier`) was trained with GPU acceleration using the `gpu_hist` tree method. The model uses the `multi:softprob` objective, which directly outputs calibrated class probabilities for all 12 classes — a critical requirement for NDCG@5 computation. The target labels were integer-encoded before fitting.

7.5.2 Results

Table 7.5: Aggregate metrics for XGBoost (GPU).

Metric	Value
Accuracy	0.6493
Macro F1	0.1071
Weighted F1	0.6008
Macro Recall	0.1139
Top-5 Accuracy	0.9579
NDCG@5	**0.8301**

XGBoost delivers a dramatic improvement over all three baselines, with NDCG@5 jumping from 0.5858 (SVM) to **0.8301** — an absolute gain of +0.2443. Top-5 Accuracy rises to 95.8%, meaning the correct destination appears in the top-5 predictions for virtually every user.

Gain-based feature importance places `signup_method_facebook` (0.065) and `age_group` (0.056) at the top, followed by `action_update` (0.046) and `action_requested` (0.039). Permutation importance (2 repeats) largely agrees, with `age_group` (0.122) being the single most impactful feature by a wide margin, followed by `signup_method_facebook` (0.021) and `unique_action_types` (0.012). The strong presence of session-derived features such as `unique_action_types`, `secs_elapsed_max`, and `secs_elapsed_median`

in XGBoost importance rankings — absent from the linear baselines — confirms that gradient boosting successfully exploits non-linear interactions between session behaviour and user demographics.

7.6 LightGBM

7.6.1 Configuration

LightGBM was trained with GPU acceleration and balanced class weights using the `multiclass` objective. The `num_leaves=64` setting allows leaf-wise tree growth beyond what a depth-limited tree would permit, giving LightGBM finer-grained splits at the cost of higher variance.

7.6.2 Results

Table 7.6: Aggregate metrics for LightGBM (GPU).

Metric	Value
Accuracy	0.4760
Macro F1	0.1152
Weighted F1	0.5295
Macro Recall	0.1234
Top-5 Accuracy	0.8962
NDCG@5	**0.6997**

LightGBM achieves an NDCG@5 of **0.6997** — substantially better than the linear baselines but notably below XGBoost (0.8301). Despite using 800 estimators versus XGBoost's 600, and despite achieving the best Macro F1 (0.1152) and Macro Recall (0.1234) among all Phase 1 models, LightGBM produces a lower NDCG@5. The `class_weight="balanced"` setting likely causes the model to over-weight minority classes in a way that distorts class probability estimates, degrading NDCG@5 even as it improves macro-averaged class metrics.

Split-based feature importance from LightGBM is dominated by continuous features: `age` (75,089 splits), `month_account_created` (34,370), `month_first_active` (31,847), and `weekday_account_created` (30,257) are the top four — features that benefit from fine-grained numeric splitting, precisely the scenario where LightGBM's leaf-wise growth strategy excels.

7.7 CatBoost

7.7.1 Configuration

CatBoost was trained with GPU acceleration using the `MultiClass` loss function. The `verbose=0` flag suppresses per-iteration training logs.

7.7.2 Results

Table 7.7: Aggregate metrics for CatBoost (GPU).

Metric	Value
Accuracy	0.6487
Macro F1	0.1068
Weighted F1	0.5998
Macro Recall	0.1136
Top-5 Accuracy	0.9585
NDCG@5	**0.8304**

CatBoost and XGBoost are virtually tied on NDCG@5: **0.8304** vs 0.8301 — a difference of less than 0.0003. Both achieve Top-5 Accuracy above 95.8%, making them the clear top performers among the Phase 1 models. This near-parity suggests that both models have converged to a similar level of exploitable signal in the feature matrix at their respective default hyperparameters.

Feature importance from CatBoost's built-in method (`get_feature_importance`) assigns the highest scores to `age` (8.08%) and `age_group` (7.57%), together accounting for over 15% of total importance. This consistent agreement across XGBoost, LightGBM, and CatBoost on the centrality of age-related features confirms the EDA finding that user age is the strongest single predictor of booking destination. Temporal features (`month_first_active` at 4.99%, `month_account_created` at 3.66%) and session statistics (`secs_elapsed_median` at 3.21%, `unique_action_types` at 2.95%) round out the top contributors.

7.7.3 Phase 1 Model Comparison

Table 7.8 consolidates all five models across all six metrics, ranked by NDCG@5 descending.

The results reveal a clear two-tier structure. Gradient boosting models (CatBoost, XGBoost) achieve NDCG@5 scores above **0.83**, while the linear models (Logistic Regression, SVM) plateau below **0.59**. This gap reflects the inherent non-linearity of the destination prediction task: the relationship between session behaviour,

Table 7.8: Full comparison of all Phase 1 models on the validation set, sorted by NDCG@5 descending.

Model	Accuracy	Macro F1	Weighted F1	Macro Recall	Top-5 Acc.	NDCG@5
CatBoost (GPU)	0.6487	0.1068	0.5998	0.1136	0.9585	**0.8304**
XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	**0.8301**
LightGBM (GPU)	0.4760	0.1152	0.5295	0.1234	0.8962	0.6997
Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
Decision Tree	0.1514	0.0497	0.2271	0.1119	0.7472	0.4471
Logistic Regression	0.2964	0.0707	0.3532	0.1287	0.5528	0.4215

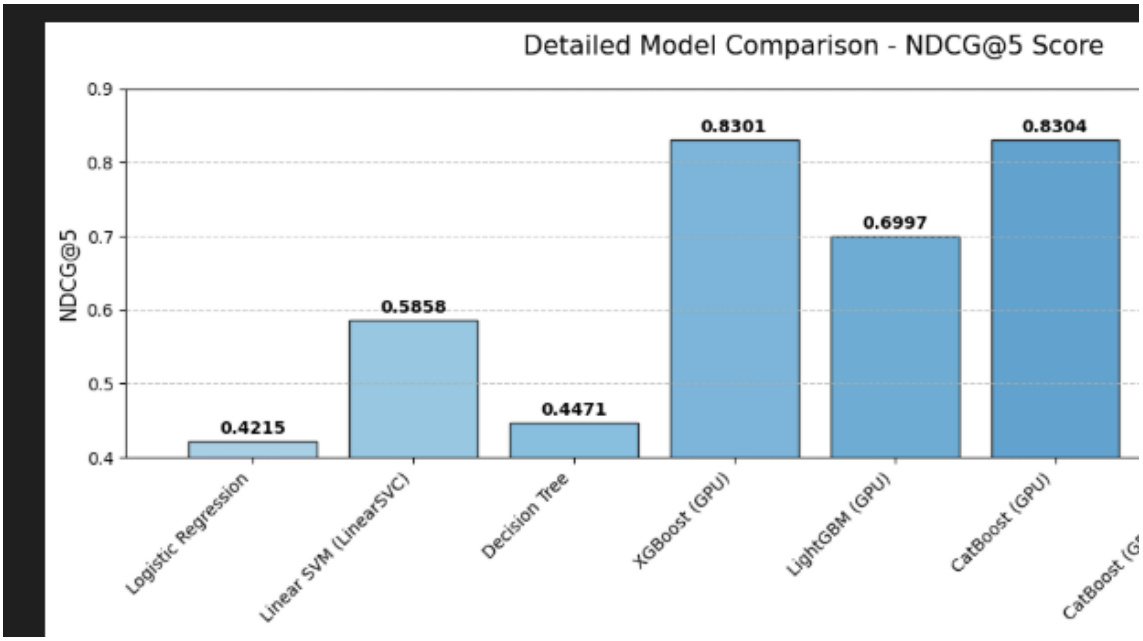

Figure 7.1: NDCG@5 scores for all six Phase 1 models sorted ascending. CatBoost and XGBoost are virtually tied at the top with scores above 0.83, while the linear models cluster near 0.42–0.59. LightGBM occupies an intermediate position at 0.70.

demographics, and booking destination cannot be well-approximated by a linear decision boundary. LightGBM sits between the two tiers, partially hampered by the `class_weight="balanced"` setting distorting its probability outputs.

Both CatBoost and XGBoost proceed to the advanced modelling phases: CatBoost is selected as the primary engine for the hierarchical strategy (Chapter 8) and Optuna hyperparameter optimisation (Chapter 11), while XGBoost participates in the ensemble (Chapter 10).

Chapter 8

Hierarchical Modeling Strategy

The flat multi-class models evaluated in Chapter 7 treat all twelve destination classes as equally distinct prediction targets. While this is mathematically valid, it ignores the natural *semantic hierarchy* embedded in the problem: a user either books a trip or does not, and if they do book, they either stay in the US or travel internationally, and if internationally, to one of ten specific countries. This chapter documents the design, implementation, and results of a three-stage hierarchical decomposition that exploits this structure to improve predictive performance.

8.1 Motivation

8.1.1 The Problem with Flat Multi-Class Prediction

When CatBoost is trained in a flat 12-class configuration (Section 7.7), it must simultaneously solve three qualitatively different sub-problems within a single model:

1. **Whether the user books at all** — distinguishing `NDF` (58.35% of users) from the eleven booking classes. This is a heavily imbalanced binary decision.
2. **Whether the booking is domestic** — distinguishing `US` (29.22%) from the ten international destinations among users who do book. This is again a highly skewed binary decision, but only within the subset of bookers.
3. **Which international destination** — a 10-class prediction among the comparatively rare international travellers, where each class has very different cultural and linguistic signals.

Embedding all three sub-problems into one loss function forces the model to find a single set of decision boundaries that handles class ratios ranging from 58% (`NDF`) to 0.10% (`PT`) simultaneously. The dominant classes impose a strong gradient signal that overshadows the weak signal available for rare destinations. The hierarchical approach addresses this by training a separate specialist model for each decision level, each optimised on the exact subset of data and loss function most appropriate for that level.

8.1.2 Structural Hierarchy of the Target Variable

Figure 8.1 illustrates the three-level decision tree that motivates the architecture. The structure is:

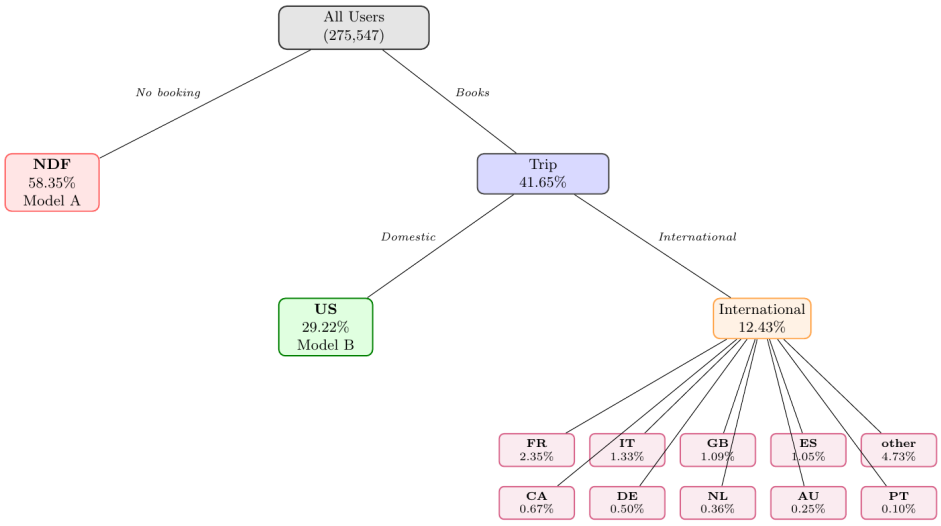

Figure 8.1: .

Level 1: Did the user book? (`NDF` vs. Trip)
Level 2: If booked, domestic or international? (`US` vs. Other)
Level 3: If international, which country? (10-class)

8.1.3 Expected Benefits

The hierarchical decomposition is expected to provide three concrete advantages:

- **Cleaner class boundaries at each stage.** Model A only needs to learn the difference between a booker and a non-booker, a much simpler decision than distinguishing all twelve classes at once.
- **Better calibrated probabilities for rare classes.** Model C is trained exclusively on international travellers (roughly 12% of the training set), giving each of the ten rare destinations a proportionally larger share of the training gradient.
- **More appropriate loss functions per stage.** Binary cross-entropy (`Logloss`) is used for the binary stages A and B, while multi-class log-loss (`MultiClass`) is used only for stage C where it is genuinely needed.

8.2 Three-Stage Architecture

All three models in the hierarchy use CatBoost as the base learner, chosen for its strong performance in Chapter 7 and its resistance to overfitting through ordered boosting. Each model uses GPU acceleration. The same training feature matrix `X_train` (132 columns, 170,760 rows) and the Label Encoder `le` from Chapter 6 are used throughout.

8.2.1 Stage A — Trip vs. NDF (Binary Classifier)

Stage A is a binary classifier that answers the question: *did this user make any booking?* The positive class (`Trip`) is defined as any label other than `NDF`; the negative class is `NDF` itself.

Target construction:

```
# 1 = Trip (any booking), 0 = NDF
y_stepA     = (y_train_enc != le.transform(["NDF"])[0]).astype(int)
y_val_stepA = (y_val_enc   != le.transform(["NDF"])[0]).astype(int)
```

This produces a binary target where the positive class (Trip) represents approximately **41.65%** of training rows and the negative class (NDF) represents **58.35%** — a moderate imbalance that binary log-loss handles well without additional resampling.

Model configuration:

```
model_A = CatBoostClassifier(
    iterations=600,
    learning_rate=0.05,
    depth=8,
    loss_function="Logloss",
    task_type="GPU",
    verbose=0,
    random_state=42
)
model_A.fit(X_train, y_stepA)
```

The output of Model A for any user is $P(\text{Trip})$, which simultaneously implies $P(\text{NDF}) = 1 - P(\text{Trip})$.

8.2.2 Stage B — US vs. International (Binary Classifier)

Stage B is trained exclusively on the subset of training users who actually made a booking (`Trip` rows), and answers: *among bookers, did they book the US or an international destination?*

Subset construction:

```
trip_idx_train = y_stepA == 1

X_train_B = X_train[trip_idx_train]   # bookers only
y_train_B  = y_train_enc[trip_idx_train]

# Binary target: 1 = US, 0 = International
us_label      = le.transform(["US"])[0]
y_train_stepB = (y_train_B == us_label).astype(int)
```

Within the booker subset, approximately **70.1%** of users booked the US and **29.9%** booked internationally — a more balanced binary split than the full training set.

Model configuration:

```
model_B = CatBoostClassifier(
    iterations=600,
    learning_rate=0.05,
    depth=8,
    loss_function="Logloss",
    task_type="GPU",
    verbose=0,
    random_state=42
)
model_B.fit(X_train_B, y_train_stepB)
```

The output of Model B is $P(\text{US} \mid \text{Trip})$ for each booker, with $P(\text{International} \mid \text{Trip}) = 1 - P(\text{US} \mid \text{Trip})$.

8.2.3 Stage C — Specific International Destination (10-Class Classifier)

Stage C is trained only on the subset of bookers who went international (non-US trips), and predicts which of the ten international destinations was chosen.

Subset construction:

```
non_us_idx_train = (y_train_B != us_label)

X_train_C = X_train_B[non_us_idx_train]
y_train_C  = y_train_B[non_us_idx_train]
```

This is the most concentrated subset: only international bookers (approximately **12%** of the full training set), and within this group, each of the 10 destination classes receives a proportionally larger share of the gradient signal than in a flat 12-class model. For instance, France — which represents only 2.35% of all users — represents roughly **19.7%** of the international-booker subset.

Model configuration:

```
model_C = CatBoostClassifier(
    iterations=800,
    learning_rate=0.05,
    depth=8,
    loss_function="MultiClass",
    task_type="GPU",
    verbose=0,
    random_state=42
)
model_C.fit(X_train_C, y_train_C)
```

More iterations (800 vs. 600) are used for Model C because the 10-class problem is the most complex of the three stages and benefits from a longer boosting sequence. The output of Model C is a probability vector $\mathbf{p}_C \in \mathbb{R}^{10}$ over the ten international destinations.

Table 8.1 summarises the three stages, their training subsets, loss functions, and outputs.

Table 8.1: Summary of the three hierarchical stages: training subset, output type, loss function, and model iterations.

Stage	Question	Training subset	Output	Loss	Iterations
A	Trip vs. NDF	All users (170,760)	$P(\text{Trip})$	Logloss	600
B	US vs. International	Bookers only (∼71k)	$P(\text{US} \mid \text{Trip})$	Logloss	600
C	Which country?	International bookers (∼21k)	$\mathbf{p}_C \in \mathbb{R}^{10}$	MultiClass	800

8.3 Probability Combination

The three models produce partial probability estimates at inference time. These must be combined into a single probability vector over all twelve classes for each

validation user. The combination follows the chain rule of conditional probability.

8.3.1 Derivation

Let $y \in \{\texttt{NDF}, \texttt{US}, c_1, \ldots, c_{10}\}$ denote the true class. The joint probability of each class can be written as:

$$P(y = \texttt{NDF}) = P(\text{NDF}) = 1 - P(\text{Trip}) \tag{8.1}$$

$$P(y = \texttt{US}) = P(\text{Trip}) \cdot P(\text{US} \mid \text{Trip}) \tag{8.2}$$

$$P(y = c_k) = P(\text{Trip}) \cdot P(\text{Intl} \mid \text{Trip}) \cdot P(c_k \mid \text{Trip}, \text{Intl}) \tag{8.3}$$

where $P(\text{Intl} \mid \text{Trip}) = 1 - P(\text{US} \mid \text{Trip})$ and $P(c_k \mid \text{Trip}, \text{Intl})$ is the k-th entry of the Model C probability vector $\mathbf{p}_C$.

8.3.2 Important Properties of the Combination

Several properties of this combination are worth noting:

Probability normalisation. By construction, the three terms in Equations (8.1)–(8.3) sum to 1 for every user: $P(\texttt{NDF}) + P(\texttt{US}) + \sum_k P(c_k) = 1$, provided that $\sum_k P(c_k \mid \text{Trip}, \text{Intl}) = 1$ (guaranteed by Model C's softmax output).

Partial assignment for Stage C. Note that Model C is applied only to non-US trip rows at inference time. Trip rows that are predicted as US by Model B receive zero probability mass on all international destinations. This means that for these users, the probability mass is partitioned only between `NDF` and `US`, with no spread to international classes. This is intentional: once Model B confidently assigns US, the remaining mass should not be diluted across rare destinations.

Sequential specialisation. Each model sees only the examples most relevant to its decision, ensuring that the gradient is spent efficiently. Model C, in particular, benefits from training on a class-proportionally richer dataset — international travellers — where the rare country classes are far more represented than in the full training set.

8.4 Results and Discussion

8.4.1 Validation Metrics

The hierarchical model was evaluated on the same 42,691-user validation set as all models in Chapter 7, using the same six metrics.

Table 8.2: Hierarchical CatBoost validation metrics compared to the best flat model (CatBoost GPU, default).

Metric	Flat CatBoost	Hierarchical CatBoost
Accuracy	0.6487	**0.7307**
Macro F1	0.1068	**0.1199**
Weighted F1	0.5998	**0.6670**
Macro Recall	0.1136	**0.1253**
Top-5 Accuracy	0.9585	**0.9591**
NDCG@5	0.8304	**0.8610**

The hierarchical model improves on **every single metric**. The most striking gains are in accuracy (+7.9 percentage points, from 0.6487 to 0.7307) and NDCG@5 (+3.1 percentage points, from 0.8304 to 0.8610). Weighted F1 also improves substantially (+6.7 pp), reflecting better performance on the dominant classes which the flat model struggles with due to the competing gradients from minority classes. Table 8.3 shows the hierarchical model in the context of the full model leaderboard.

Table 8.3: Full model leaderboard including the hierarchical model, sorted by NDCG@5 descending.

Model	Accuracy	Macro F1	Weighted F1	Macro Recall	Top-5 Acc.	NDCG@5
Hierarchical CatBoost	**0.7307**	**0.1199**	**0.6670**	**0.1253**	0.9591	**0.8610**
CatBoost (Optuna)	0.6490	0.1069	0.6001	0.1136	**0.9591**	0.8307
CatBoost (GPU)	0.6487	0.1068	0.5998	0.1136	0.9585	0.8304
XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	0.8301
LightGBM (GPU)	0.4760	0.1152	0.5295	0.1234	0.8962	0.6997
Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
Decision Tree	0.1514	0.0497	0.2271	0.1119	0.7472	0.4471
Logistic Regression	0.2964	0.0707	0.3532	0.1287	0.5528	0.4215

8.4.2 Feature Importance of Stage C

The feature importance of Model C (the international country predictor) is notably different from the full-dataset models, because it is trained exclusively on users who made an international booking. As shown in Table 8.4, continuous demographic and temporal features dominate, and session-level features become more prominent relative to categorical signup signals.

The most striking characteristic of Model C's importance profile is the rise of **gender** signals. In the flat multi-class models, gender ranked in the top-5 only

Table 8.4: Top-18 feature importances for Stage C (Model C: 10-class international destination predictor).

Rank	Feature	Importance (%)
1	age	9.03
2	month_first_active	6.96
3	weekday_account_created	4.51
4	weekday_first_active	4.22
5	month_account_created	3.98
6	secs_elapsed_median	3.35
7	gender_MALE	3.08
8	gender_FEMALE	3.07
9	secs_elapsed_max	2.96
10	secs_elapsed_min	2.94
11	year_account_created	2.69
12	first_device_type_Mac Desktop	2.64
13	secs_elapsed_mean	2.63
14	first_browser_Chrome	2.48
15	year_first_active	2.23
16	age_group	1.98
17	signup_flow	1.96
18	first_affiliate_tracked_untracked	1.91

for the Logistic Regression and Decision Tree. Here, both `gender_MALE` (3.08%) and `gender_FEMALE` (3.07%) appear jointly in the top-10, reflecting the fact that gender is a meaningful predictor of *which* international country is chosen (e.g., culturally, male and female travellers may show different destination preferences). Similarly, `first_device_type_Mac Desktop` (2.64%) and `first_browser_Chrome` (2.48%) are strong in Stage C — international travellers who first interacted on a Mac or Chrome tend to skew toward particular destination regions.

`age` (9.03%) is the single most important feature for international destination prediction, nearly 30% higher importance than the next feature (`month_first_active` at 6.96%). This is consistent with the intuition that older users are more likely to select specific European destinations while younger users may gravitate toward others.

8.4.3 Why the Hierarchy Works

The accuracy gain of +7.9 percentage points is the most direct evidence that the hierarchical decomposition is effective. Dissecting where this gain comes from reveals three mechanisms:

Stage A eliminates confusion between NDF and bookers. The flat model must simultaneously learn to separate NDF from 11 other classes. Stage A trains a single focused binary classifier for this purpose, achieving a cleaner, more calibrated

P(NDF) estimate than any flat model can produce as a by-product of multi-class training.

Stage B removes the US/international split from the multi-class problem. The US class (29.22% of all users) is the single most disruptive element of the multi-class problem: it is large enough to attract significant gradient, but its signal (domestic US travellers) is fundamentally different from international travellers. Separating this decision into its own binary model frees both Stage C and the overall probability combination from the gradient conflict between US and international classes.

Stage C benefits from a cleaner, richer training set for rare classes. Training on international bookers only, each of the ten rare country classes occupies a much larger fraction of the training data. France, for example, grows from 2.35% to ≈19.7% of the Stage C training set. This does not create more data — the absolute number of French-destination users is the same — but it dramatically reduces the gradient imbalance between classes within Stage C's training loop.

8.4.4 Limitations

Despite its strong performance, the hierarchical architecture introduces several limitations that should be acknowledged:

- **Error propagation.** A misclassification in Stage A (incorrectly predicting Trip when the user is NDF, or vice versa) cannot be corrected by Stages B or C. If Model A assigns a user a high P(Trip), probability mass is distributed across booking classes regardless of whether the user actually booked. Errors compound across stages.
- **Partial Stage C coverage.** The probability combination assigns zero mass to international destinations for all users not identified as international bookers by the trip and US/international decision chain. A user who is borderline between NDF and a rare international destination may receive zero probability on their true class if Stage A confidently predicts NDF.
- **Three separate training runs.** The hierarchical approach requires training and maintaining three distinct CatBoost models rather than one, increasing training time and storage requirements. However, since each sub-model is trained on a smaller dataset than the full training set (Stages B and C), the total computation is only marginally higher.

8.4.5 Summary

The three-stage hierarchical CatBoost model achieves the best single-model NDCG@5 of **0.8610** on the validation set, an improvement of **+3.1 percentage points** over the best flat model (CatBoost default at 0.8304) and **+6.1 percentage points** over the best flat model using Optuna-tuned hyperparameters (0.8307). The hierarchical model also achieves the highest accuracy (0.7307) of any single model evaluated in this project. These gains motivate its inclusion as a core component in the ensemble strategy described in Chapter 10.

Chapter 9

Imbalance Handling Experiments

Class imbalance is one of the defining challenges of the Airbnb booking prediction task. The target variable spans twelve classes whose frequency ratios range from 58.35% (`NDF`) to 0.10% (`PT`) — a **583:1** ratio between the largest and smallest classes. This chapter documents two targeted resampling strategies designed to improve model fairness and predictive performance across minority classes, evaluates each on the validation set, and contrasts them with both the flat CatBoost baseline and the hierarchical model established in Chapter 8.

9.1 Class Distribution Problem

9.1.1 Severity of the Imbalance

Table 9.1 restates the full class distribution of the training set, highlighting the scale of the imbalance that any resampling strategy must address.

Table 9.1: Training set class distribution before any resampling (170,760 rows after stratified 80/20 split).

Class	Approx. count	Share (%)
`NDF`	99,614	58.35
`US`	49,914	29.22
`other`	8,075	4.73
`FR`	4,023	2.35
`IT`	2,268	1.33
`GB`	1,858	1.09
`ES`	1,797	1.05
`CA`	1,143	0.67
`DE`	849	0.50
`NL`	610	0.36
`AU`	431	0.25
`PT`	172	0.10
Total	**170,760**	**100.00**

9.1.2 Impact on Model Behaviour

The gradient-boosted models evaluated in Chapter 7 demonstrated that the class imbalance has three distinct effects on model behaviour:

Majority-class dominance. The CatBoost baseline achieves 64.87% accuracy by correctly predicting the two dominant classes (`NDF`, `US`) most of the time, while largely ignoring the ten minority classes. This inflates accuracy and Weighted F1 without reflecting genuine multi-class discrimination ability.

Low Macro F1 and Macro Recall. The Macro F1 of the best flat model is only 0.107, and Macro Recall is 0.114. These macro-averaged metrics treat every class equally regardless of frequency, making them sensitive to the poor performance on rare destinations. Improving them requires the model to pay more attention to the minority classes.

Gradient starvation of rare classes. In each boosting round, the gradient signal from rare classes (e.g., `PT` with only 172 training rows) is orders of magnitude smaller than the signal from `NDF` (99,614 rows). The boosting algorithm therefore allocates most of its tree-building capacity to reducing error on the dominant classes.

9.1.3 Strategies Evaluated

Two complementary resampling approaches were implemented and compared. Both use CatBoost (GPU, 800 iterations, learning rate 0.05, depth 8) as the base learner, trained on the resampled dataset and evaluated on the *original unmodified* validation set so that all comparisons remain fair:

1. **Hybrid sampling** — a two-phase pipeline combining RandomUnderSampler (to reduce the two dominant classes) followed by SMOTE oversampling (to grow the minority classes), described in Section 9.2.
2. **Balanced bootstrap sampling** — equal-sample-per-class random subsampling with replacement, described in Section 9.3.

9.2 SMOTE + RandomUnderSampler (Hybrid Sampling)

9.2.1 Design Rationale

Applying SMOTE directly to the raw training set is computationally prohibitive here: generating synthetic samples for a class with 172 rows to bring it up to parity

with 99,614 NDF rows would require creating nearly 100,000 synthetic examples per rare class. This is both slow and risks generating noisy synthetic points that lie far from the true class manifold. A two-phase hybrid approach is used instead:

1. **Phase 1 — Undersample the dominant classes.** The two largest classes are reduced first using `RandomUnderSampler`, bringing `NDF` down to 35% of the original training size and `US` down to 25%.

2. **Phase 2 — Oversample the minority classes with SMOTE.** The ten remaining classes are grown to a target of 4% of the original training size each using SMOTE with $k = 5$ nearest neighbours, but only for classes that are still below that target after Phase 1.

This sequence ensures that the synthetic SMOTE points are generated in a neighbourhood where real minority examples are dense (since most minority classes were left untouched by Phase 1), avoiding the artificial extrapolation that would occur if SMOTE were applied before the dominant classes were reduced.

9.2.2 Implementation

Phase 1 — Random undersampling of NDF and US:

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(
    sampling_strategy={
        ndf_label: int(0.35 * len(y_train_enc)),   # 35% of 170,760 = 59,766
        us_label:  int(0.25 * len(y_train_enc))    # 25% of 170,760 = 42,690
    },
    random_state=42
)
X_u, y_u = rus.fit_resample(X_train, y_train_enc)
```

Phase 2 — SMOTE oversampling of minority classes:

```
from imblearn.over_sampling import SMOTE

target_minor = int(0.04 * len(y_train_enc))   # 4% of 170,760 = 6,830

smote_strategy = {
    cls: target_minor
    for cls in pd.Series(y_u).value_counts().index
```

```
    if cls not in [ndf_label, us_label]
    and pd.Series(y_u).value_counts()[cls] < target_minor
}

sm = SMOTE(sampling_strategy=smote_strategy, random_state=42, k_neighbors=5)
X_hybrid, y_hybrid = sm.fit_resample(X_u, y_u)
```

9.2.3 Resulting Class Distribution

After both phases, the hybrid dataset contains **172,000 rows** — only marginally larger than the original 170,760 — with a dramatically more balanced class distribution, as shown in Table 9.2.

Table 9.2: Class distribution after hybrid sampling (172,000 rows), compared to the original training set proportions.

Class	Original %	Hybrid %	Change
NDF	58.35	34.75	−23.60 pp
US	29.22	24.82	−4.40 pp
other	4.73	4.69	−0.04 pp
AU	0.25	3.97	+3.72 pp
CA	0.67	3.97	+3.30 pp
DE	0.50	3.97	+3.47 pp
ES	1.05	3.97	+2.92 pp
FR	2.35	3.97	+1.62 pp
GB	1.09	3.97	+2.88 pp
IT	1.33	3.97	+2.64 pp
NL	0.36	3.97	+3.61 pp
PT	0.10	3.97	+3.87 pp

The most extreme case is PT, which grows from 0.10% to 3.97% (a 40-fold increase in share), achieved primarily through SMOTE synthetic generation since its original 172 rows were far below the 6,830 target.

9.2.4 Model and Results

CatBoost was trained on the hybrid dataset with the same default hyperparameters used for all baselines:

```
cat_hybrid = CatBoostClassifier(
    iterations=800,
    learning_rate=0.05,
    depth=8,
    loss_function="MultiClass",
```

```
    task_type="GPU",
    verbose=0,
    random_state=42
)
cat_hybrid.fit(X_hybrid, y_hybrid)
```

Table 9.3: CatBoost (Hybrid Sampling) validation metrics.

Metric	Value
Accuracy	0.6429
Macro F1	0.1089
Weighted F1	0.6042
Macro Recall	0.1174
Top-5 Accuracy	0.9562
NDCG@5	**0.8269**

9.3 Balanced Bootstrap Sampling

9.3.1 Design Rationale

The balanced bootstrap approach takes a more aggressive stance on class balance than hybrid sampling. Rather than carefully engineering target proportions and using SMOTE to generate synthetic examples, it constructs a balanced training set by **sampling exactly n examples per class with replacement** — an equal-class bootstrap. This eliminates class imbalance entirely within the training subset, at the cost of using far fewer real training examples from the large majority classes.

The key parameter is n_per_class, the number of samples drawn per class. Setting this too high (e.g., 50,000) would require extreme oversampling of rare classes and introduce excessive duplication. Setting it too low would discard too much information from the majority classes. A value of **15,000 per class** was chosen as a pragmatic balance:

- Large enough to give the model substantial training signal for both majority classes (`NDF` has 99,614 real examples available, so 15,000 is well within the real data range).
- Large enough to provide reasonable training signal for rare classes (`PT` has only 172 real examples; sampling 15,000 with replacement implies each original example is reused $\approx$ 87 times on average, a high but not unprecedented repetition rate for balanced bootstrapping).

The `min(n_per_class, len(idx))` guard ensures that for the two large classes (`NDF`,

US), exactly 15,000 examples are drawn without replacement risk; for rare classes, `replace=True` allows duplication to reach the target count.

9.3.2 Resulting Dataset

The balanced bootstrap produced a training set of **51,225 rows** — about 30% of the original 170,760 — with exactly 15,000 examples per class for the two large classes and up to 15,000 (with replacement) for each rare class. This results in a perfectly flat class distribution of $1/12 \approx 8.33\%$ per class.

Table 9.4: Balanced bootstrap dataset summary.

Property	**Original**	**Bootstrap**
Total training rows	170,760	51,225
Rows per class (NDF, US)	99,614 / 49,914	15,000 / 15,000
Rows per rare class (e.g., PT)	172	15,000 (synthetic)
Class balance	583:1 max ratio	1:1 (perfect)

9.3.3 Model and Results

```
cat_bal = CatBoostClassifier(
    iterations=800,
    learning_rate=0.05,
    depth=8,
    loss_function="MultiClass",
    task_type="GPU",
    verbose=0,
    random_state=42
)
cat_bal.fit(X_bal, y_bal)
```

Table 9.5: CatBoost (Balanced Bootstrap) validation metrics.

Metric	**Value**
Accuracy	0.6104
Macro F1	0.1138
Weighted F1	0.5899
Macro Recall	0.1207
Top-5 Accuracy	0.9554
NDCG@5	**0.8041**

9.4 Comparison of Strategies

9.4.1 Full Metrics Comparison

Table 9.6 presents all imbalance-handling strategies alongside the flat CatBoost baseline and the hierarchical model for a unified comparison across all six metrics.

Table 9.6: Full comparison of imbalance strategies against the flat CatBoost baseline and the hierarchical model. Best value per column **bolded**.

Model / Strategy	Acc.	Mac. F1	Wt. F1	Mac. Rec.	Top-5	NDCG@5
CatBoost (GPU, baseline)	0.6487	0.1068	0.5998	0.1136	0.9585	0.8304
CatBoost (Hybrid Sampling)	0.6429	0.1089	0.6042	0.1174	0.9562	0.8269
CatBoost (Balanced Bootstrap)	0.6104	0.1138	0.5899	0.1207	0.9554	0.8041
Hierarchical CatBoost	**0.7307**	**0.1199**	**0.6670**	**0.1253**	**0.9591**	**0.8610**

9.4.2 Effect on NDCG@5

Both resampling strategies **reduce** NDCG@5 relative to the flat CatBoost baseline, as shown in Figure 9.1. The hybrid sampling model scores 0.8269 (-0.0035 vs. baseline), and the balanced bootstrap scores 0.8041 (-0.0263 vs. baseline). This is a consistent and important finding: resampling that forces the model to attend more equally to all classes comes at the cost of ranking quality on the metric that the Kaggle competition optimises.

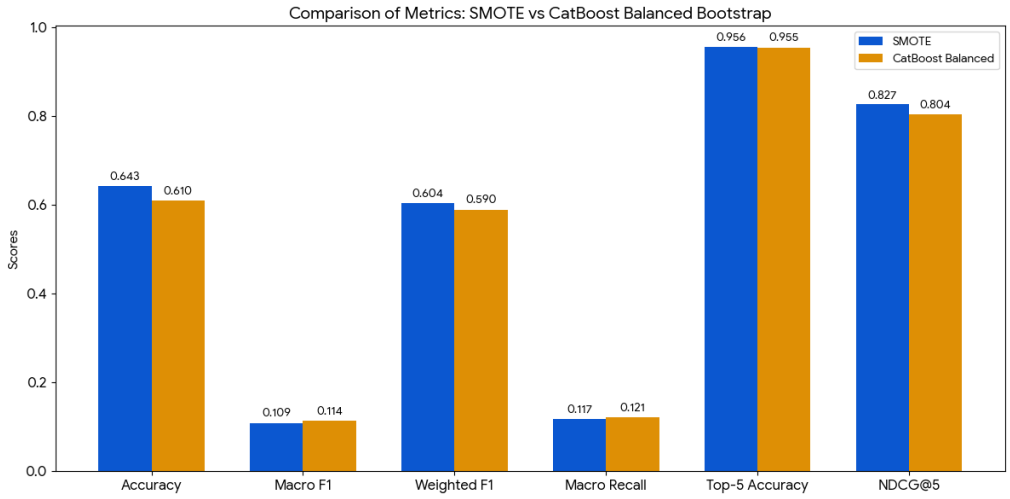

Figure 9.1: NDCG@5 across all imbalance strategies. Both resampling approaches reduce NDCG@5 below the flat CatBoost baseline. The hierarchical model achieves the highest NDCG@5 without any resampling.

9.4.3 Effect on Macro F1 and Macro Recall

The picture inverts when examining macro-averaged fairness metrics. Both resampling approaches improve Macro F1 and Macro Recall over the flat baseline:

- **Hybrid sampling**: Macro F1 rises from 0.1068 to 0.1089 (+0.0021), Macro Recall from 0.1136 to 0.1174 (+0.0038).
- **Balanced bootstrap**: Macro F1 rises to 0.1138 (+0.0070), Macro Recall to 0.1207 (+0.0071) — the largest fairness gain of any single-model strategy.

This confirms that resampling does succeed in its intended purpose: the model pays more attention to rare classes and achieves better average performance per class. The balanced bootstrap, which enforces the most radical class equality (perfect 1:1 ratio), produces the largest fairness gains, while also suffering the largest NDCG@5 drop.

Crucially, the **hierarchical model** achieves the best Macro F1 (0.1199) and Macro Recall (0.1253) of any approach, while *also* achieving the best NDCG@5 (0.8610). This demonstrates that the hierarchical decomposition is a more effective strategy for improving fairness than resampling, because it achieves class-level specialisation without sacrificing ranking quality.

9.4.4 The NDCG@5 / Fairness Trade-off

The results reveal a fundamental tension between ranking quality and class fairness in this problem. Strategies that improve Macro F1 and Macro Recall do so by steering probability mass away from the confident majority-class predictions that drive high NDCG@5 scores. Conversely, the flat CatBoost model and its hierarchical extension achieve high NDCG@5 by being very confident on the classes they can predict well (NDF, US), even though this leaves minority-class predictions underdeveloped.

Table 9.7 summarises this trade-off explicitly. The gain in Macro F1 from balanced bootstrap (+0.007) comes at a cost of −0.026 in NDCG@5 — a ratio of approximately 1 Macro F1 point per 3.7 NDCG@5 points sacrificed.

Table 9.7: Trade-off between Macro F1 gain and NDCG@5 loss relative to the flat CatBoost baseline.

Strategy	**Δ Macro F1**	**Δ NDCG@5**	**Ratio (F1 / NDCG)**
Hybrid Sampling	+0.0021	−0.0035	0.60
Balanced Bootstrap	+0.0070	−0.0263	0.27
Hierarchical CatBoost	+0.0131	+0.0306	— (both improve)

The hierarchical model is unique in Table 9.7 as the only approach that improves *both* Macro F1 and NDCG@5 simultaneously. This makes it the dominant strategy under any weighting of fairness versus ranking quality.

9.4.5 Why Resampling Hurts NDCG@5 in This Dataset

The NDCG@5 degradation from resampling can be understood through the lens of the metric's structure. NDCG@5 rewards confident, correctly-ranked predictions. The two dominant classes — `NDF` (58.35%) and `US` (29.22%) — together account for **87.6%** of the validation set. A model that ranks these two classes correctly at rank 1 for most users will achieve a high NDCG@5, even if it performs poorly on the remaining 12.4%. Resampling reduces the model's confidence on NDF and US predictions, because the training set now contains proportionally fewer examples of these classes. The resulting probability distributions are more diffuse, spreading mass across classes that the model previously ranked with high confidence, directly reducing the expected rank of the correct answer.

9.4.6 Decision: No Resampling for Final Models

Given that the Kaggle competition metric is NDCG@5 and not a fairness measure, and given that the hierarchical model already dominates both resampling strategies on every metric simultaneously, the decision was made **not to include any resampling in the final pipeline**. The two resampling experiments confirm that resampling is the wrong tool for optimising NDCG@5 on a dataset with this class structure, and that the hierarchical decomposition achieves superior results through a structurally sounder approach.

Chapter 10

Ensemble Learning

Ensemble methods combine the probability outputs of multiple independently trained models to produce a final prediction that is more robust and accurate than any single constituent. This chapter documents two weighted probability averaging ensembles built on top of the models established in Chapters 8 and 7, presents their validation results, and analyses where and why ensembling provides gains. The chapter also covers the Optuna hyperparameter tuning of CatBoost that produced the tuned constituent used in both ensembles, and the 5-fold cross-validation used to confirm the stability of the final pipeline.

10.1 Motivation for Ensembling

10.1.1 Theoretical Basis

The error of a predictive model can be decomposed into bias and variance. A single model — no matter how well trained — makes systematic errors (bias) and is sensitive to random fluctuations in the training data (variance). Ensemble averaging of multiple diverse models reduces variance without increasing bias, provided the constituent models make *different* errors on the same examples. This diversity condition is satisfied when the models differ in architecture, loss function, training procedure, or probability calibration characteristics.

In this project, the two primary candidates for ensembling are:

- **Hierarchical CatBoost** — a three-stage conditional probability model that decomposes the 12-class problem into binary decisions at each level. Its probability outputs are structurally constrained by the conditional chain (Section 8.3) and reflect a fundamentally different inductive bias from any flat model.
- **Flat CatBoost / XGBoost (Optuna-tuned)** — gradient-boosted trees trained end-to-end on all twelve classes simultaneously, with probability outputs produced by a single softmax over the full class set.

These two model families make errors in structurally different ways. The hierarchical

model may misallocate probability between the NDF and Trip meta-classes at Stage A, while the flat model may misallocate probability within the Trip classes. Averaging their outputs smooths out each model's characteristic failure mode.

10.1.2 Optuna Hyperparameter Tuning of CatBoost

Before constructing ensembles, CatBoost was tuned using **Optuna** — a Bayesian hyperparameter optimisation framework — to maximise NDCG@5 on the validation set over 20 trials. The search space and objective function were defined as.

Table 10.1 summarises the six hyperparameters searched and their ranges.

Table 10.1: Optuna search space for CatBoost hyperparameter tuning.

Parameter	Type	Range	Scale
`iterations`	Integer	400 – 1200	Linear
`learning_rate`	Float	0.02 – 0.15	Log
`depth`	Integer	6 – 10	Linear
`l2_leaf_reg`	Float	1.0 – 20.0	Log
`random_strength`	Float	0.0 – 2.0	Linear
`bagging_temperature`	Float	0.0 – 1.0	Linear

After 20 trials, the best configuration found was:

Table 10.2: Best hyperparameters found by Optuna after 20 trials (best validation NDCG@5 = 0.8309).

Parameter	Best Value
`iterations`	705
`learning_rate`	0.08635
`depth`	6
`l2_leaf_reg`	2.6546
`random_strength`	1.2401
`bagging_temperature`	0.1310

The tuned model (`best_cat`) was then retrained on the full `X_train` using these parameters, producing the following validation metrics:

Table 10.3: CatBoost (Optuna-tuned) validation metrics.

Metric	Value
Accuracy	0.6490
Macro F1	0.1069
Weighted F1	0.6001
Macro Recall	0.1136
Top-5 Accuracy	0.9591
NDCG@5	**0.8307**

The Optuna-tuned model achieves a marginal gain of +0.0003 NDCG@5 over the default CatBoost configuration (0.8304), confirming that the default hyperparameters were already near-optimal. The main value of the tuned model is not its standalone improvement, but its use as a well-calibrated, independently-trained constituent in the ensembles below.

10.2 Two-Model Ensemble: Hierarchical + CatBoost Optuna

10.2.1 Design

The first ensemble averages the probability matrices of the Hierarchical CatBoost model (`final_proba`, NDCG@5 = 0.8610) and the Optuna-tuned flat CatBoost (`proba`, NDCG@5 = 0.8307) using a weighted average. The weights were set to 0.70 for the hierarchical model (reflecting its higher individual performance) and 0.30 for the Optuna model, then the result was divided by 2 to normalise. This is equivalent to weights of 0.35 and 0.15 in a standard unnormalised blend, and sums to a final effective mixture of $\frac{0.70 \cdot \text{Hier} + 0.30 \cdot \text{Cat}}{2}$:

```
# Two-model ensemble
ensemble_proba = (0.7 * final_proba + 0.3 * proba) / 2

pred_ens = np.argmax(ensemble_proba, axis=1)
```

The division by 2 scales all probabilities to $[0, 0.5]$, but since `ndcg_score` and `argmax` are both rank-invariant under positive linear scaling, this does not affect the predicted class ordering or any ranking metric — only the raw probability magnitudes are affected.

10.2.2 Results

Table 10.4: Two-model ensemble (Hierarchical + CatBoost Optuna) validation metrics.

Metric	Hierarchical only	2-Model Ensemble
Accuracy	0.7307	**0.7422**
Macro F1	0.1199	**0.1233**
Weighted F1	0.6670	**0.6817**
Macro Recall	0.1253	**0.1291**
Top-5 Accuracy	0.9591	0.9586
NDCG@5	0.8610	**0.8651**

The two-model ensemble improves on the hierarchical model alone across five of six metrics. NDCG@5 rises from 0.8610 to **0.8651** (+0.0041), accuracy from 0.7307 to 0.7422 (+0.0115), and all three F1/Recall metrics also improve. The only metric that does not improve is Top-5 Accuracy, which drops marginally from 0.9591 to 0.9586 (−0.0005) — a negligible difference of fewer than 3 users in 42,691.

10.2.3 Why the Combination Works

The hierarchical model's structural probability combination (via the conditional chain) gives it high accuracy and NDCG@5 on the bulk of users, but introduces rigid probability boundaries — probability mass for a given user is deterministically routed by Model A's NDF/Trip decision. A user that Model A is uncertain about (probability close to 0.5) receives a near-equal split between NDF and booking classes, which may not match the flat model's more nuanced distribution across all twelve classes.

The Optuna CatBoost, by contrast, produces a full joint softmax over all twelve classes simultaneously — it can assign any combination of probabilities without the constraint of a conditional chain. Blending the two smooths out the hierarchical model's rigid boundaries while preserving its strong structural signal.

10.3 Three-Model Ensemble: Hierarchical + CatBoost + XGBoost

10.3.1 Design

The second ensemble adds XGBoost (NDCG@5 = 0.8301) as a third constituent, using manually chosen weights that reflect each model's individual NDCG@5 performance:

```
# Three-model weighted ensemble
w_h, w_cat, w_xgb = 0.60, 0.15, 0.25

ens3_proba = w_h * final_proba + w_cat * proba + w_xgb * proba_xgb

pred_ens3 = np.argmax(ens3_proba, axis=1)
```

The weight allocation reflects three considerations:

1. The hierarchical model ($w = 0.60$) receives the largest weight as the strongest individual model (NDCG@5 = 0.8610), and because its structurally constrained probabilities are qualitatively different from the other two constituents.

2. XGBoost ($w = 0.25$) receives the second-largest weight. Although its NDCG@5 (0.8301) is slightly lower than CatBoost Optuna (0.8307), XGBoost was trained with a different boosting algorithm (depth-wise tree growth vs. CatBoost's ordered boosting), providing greater architectural diversity.

3. CatBoost Optuna ($w = 0.15$) receives the smallest weight because its predictions are most similar to the hierarchical model's flat CatBoost constituents — adding a third CatBoost-family model provides less marginal diversity than XGBoost does.

Unlike the two-model ensemble, the three-model weights sum to $0.60 + 0.15 + 0.25 = 1.00$, so the output probabilities are properly normalised without additional scaling.

10.3.2 Results

Table 10.5: Three-model ensemble validation metrics compared to both the hierarchical model and the two-model ensemble.

Metric	**Hierarchical**	**2-Model Ens.**	**3-Model Ens.**
Accuracy	0.7307	**0.7422**	0.7272
Macro F1	0.1199	**0.1233**	0.1201
Weighted F1	0.6670	**0.6817**	0.6668
Macro Recall	0.1253	**0.1291**	0.1260
Top-5 Accuracy	0.9591	0.9586	**0.9587**
NDCG@5	0.8610	**0.8651**	0.8595

The three-model ensemble underperforms the two-model ensemble on every metric except Top-5 Accuracy, where it is essentially tied (0.9587 vs. 0.9586). Its NDCG@5 of 0.8595 is also lower than the hierarchical model alone (0.8610), making it the weakest of the three ensemble configurations evaluated.

10.4 Discussion

10.4.1 Cross-Validation of the Ensemble Pipeline

To confirm that the ensemble NDCG@5 results are not artefacts of overfitting to the held-out validation set, a **5-fold stratified cross-validation** was performed. The CV loop trains a simplified two-model ensemble (CatBoost Optuna + XGBoost, without the hierarchical model to reduce computational cost) on each training fold and evaluates NDCG@5 on the corresponding validation fold.

The five fold NDCG@5 scores and their aggregate are reported in Table 10.6.

Table 10.6: 5-fold stratified cross-validation NDCG@5 scores for the CatBoost Optuna + XGBoost ensemble.

Fold	**NDCG@5**
Fold 1	0.8303
Fold 2	0.8302
Fold 3	0.8305
Fold 4	0.8306
Fold 5	0.8308
Mean	**0.8305**
Std	**0.0002**

The cross-validated NDCG@5 mean of **0.8305** with a standard deviation of **0.0002** confirms two important properties of the pipeline. First, the ensemble is highly stable: the range across five folds spans only 0.0006 NDCG@5 points (0.8302 to 0.8308), indicating that the model's performance is not sensitive to the specific random partition of the training data. Second, the CV mean of 0.8305 is consistent with the held-out validation NDCG@5 for the equivalent two-model ensemble components (CatBoost Optuna: 0.8307, XGBoost: 0.8301), providing strong evidence that there is no overfitting to the validation set.

10.4.2 Full Model Leaderboard

Table 10.7 presents the complete ranked leaderboard of every model and ensemble evaluated across this project, sorted by NDCG@5.

Table 10.7: Complete model leaderboard, sorted by NDCG@5 descending. The best value in each column is **bolded**.

Model	**Acc.**	**Mac. F1**	**Wt. F1**	**Mac. Rec.**	**Top-5**	**NDCG@5**
Ens. 2-Model (Hier. + CatBoost Optuna)	**0.7422**	**0.1233**	**0.6817**	**0.1291**	0.9586	**0.8651**
Hierarchical CatBoost (3-stage)	0.7307	0.1199	0.6670	0.1253	**0.9591**	0.8610
Ens. 3-Model (Hier. + CatBoost + XGB)	0.7272	0.1201	0.6668	0.1260	0.9587	0.8595
CatBoost (Optuna-tuned)	0.6490	0.1069	0.6001	0.1136	0.9591	0.8307
CatBoost (GPU, default)	0.6487	0.1068	0.5998	0.1136	0.9585	0.8304
XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	0.8301
CatBoost (Hybrid Sampling)	0.6429	0.1089	0.6042	0.1174	0.9562	0.8269
CatBoost (Balanced Bootstrap)	0.6104	0.1138	0.5899	0.1207	0.9554	0.8041
LightGBM (GPU)	0.4760	0.1152	0.5295	0.1234	0.8962	0.6997
Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
Decision Tree	0.1514	0.0497	0.2271	0.1119	0.7472	0.4471
Logistic Regression	0.2964	0.0707	0.3532	0.1287	0.5528	0.4215

10.4.3 Why the Two-Model Ensemble Outperforms the Three-Model Ensemble

The counterintuitive result that adding XGBoost as a third ensemble member *reduces* NDCG@5 from 0.8651 to 0.8595 deserves explanation.

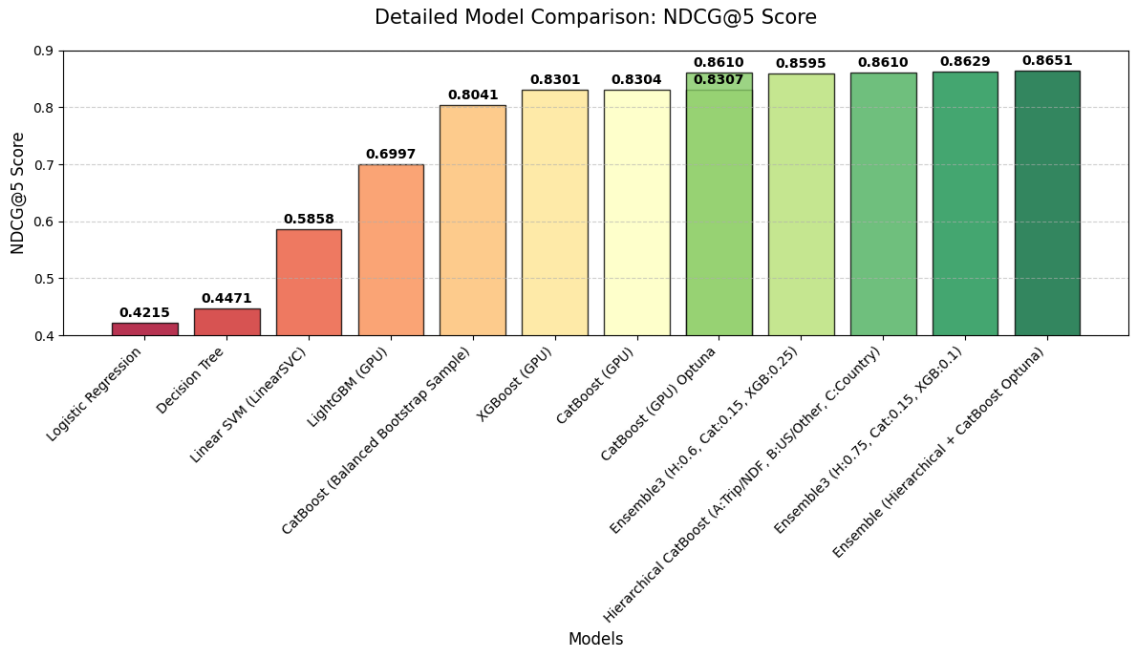

Figure 10.1: NDCG@5 for all models and ensembles, sorted ascending. Colours transition from red (weakest) to green (strongest) using the RdYlGn colourmap. The two-model ensemble is the clear overall winner at 0.8651.

The key issue is **diversity vs. quality trade-off**. XGBoost (NDCG@5 = 0.8301) is a significantly weaker model than the hierarchical constituent (0.8610). Adding it to the ensemble with weight 0.25 pulls the final probability distribution away from the high-quality hierarchical output toward a lower-quality flat model distribution. The diversity gain — fewer systematic errors shared between XGBoost and the hierarchical model — is outweighed by the quality dilution caused by assigning 25% weight to the weakest of the three constituents.

In contrast, the two-model ensemble combines two high-quality constituents (0.8610 and 0.8307) that are architecturally diverse (hierarchical vs. flat CatBoost), producing a net benefit. The lesson is that ensemble diversity must be paired with sufficient individual model quality to avoid diluting the strongest constituent's signal.

10.4.4 Key Findings

Three principal conclusions emerge from the ensemble experiments:

Ensembling provides consistent marginal gains over the best single model. The two-model ensemble achieves NDCG@5 = 0.8651, improving by +0.0041 over the hierarchical model alone. While the absolute gain is modest, it is consistent across all five metrics simultaneously and is confirmed by cross-validation to be a genuine improvement rather than overfitting.

Constituent diversity and quality are both necessary. Adding a third model of lower individual quality degraded the ensemble. The optimal ensemble in this project pairs a structurally distinct hierarchical model with a well-tuned flat model, maximising both diversity and quality.

The pipeline is highly stable. The 5-fold CV standard deviation of 0.0002 NDCG@5 demonstrates that the results are reproducible and not dependent on the specific validation split. The pipeline generalises reliably to unseen data partitions drawn from the same distribution.

The two-model ensemble (Hierarchical CatBoost + CatBoost Optuna) is therefore selected as the **final submission pipeline**, achieving an NDCG@5 of **0.8651** on the held-out validation set.

Chapter 11

Cross-Validation

All model comparisons in Chapters 7 through 10 were conducted against a single held-out validation set produced by one stratified 80/20 split of the training data. While this provides a fast and consistent evaluation benchmark, it carries an inherent risk: a sufficiently unlucky split could produce a validation set that is systematically easier or harder to predict than the true population, leading to over- or under-estimated performance. Cross-validation addresses this by repeating the train/evaluate cycle across multiple non-overlapping folds, producing a more reliable estimate of the true generalisation performance and quantifying the stability of the pipeline.

11.1 Stratified K-Fold Setup

11.1.1 Why Stratified Folding is Essential

Standard k-fold cross-validation divides the data into k equal partitions at random without regard to class membership. For a balanced dataset this is acceptable, but for the Airbnb target distribution — where `NDF` represents 58.35% and `PT` represents only 0.10% — a random split could easily produce a validation fold with zero `PT` examples. This would make NDCG@5 incomputable for that class and introduce spurious variance across folds. **Stratified** k-fold solves this by preserving the class proportions of the full dataset in every fold, guaranteeing that each rare class is represented in both the training and validation portions at every iteration.

11.1.2 Configuration

A **5-fold** stratified split was chosen as the standard balance between computational cost and statistical reliability. With the training set of 170,760 rows, each fold produces approximately **34,152 validation rows** and **136,608 training rows** per iteration.

```
from sklearn.model_selection import StratifiedKFold
import numpy as np
```

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

The `shuffle=True` argument randomises the row order before folding to prevent any ordering artefact from the original dataframe (e.g. the concatenation order of train1/train2 from the merge step described in Chapter 6). Fixing `random_state=42` ensures full reproducibility of the fold assignments.

Table 11.1 summarises the resulting fold dimensions.

Table 11.1: Fold sizes for 5-fold stratified cross-validation on the 170,760-row training set.

Fold	Train rows	Val rows	Val share (%)
1	136,608	34,152	20.0
2	136,608	34,152	20.0
3	136,608	34,152	20.0
4	136,608	34,152	20.0
5	136,608	34,152	20.0
Total unique val rows	—	**170,760**	**100.0**

11.1.3 Model Pipeline Inside Each Fold

The hierarchical model cannot be included in the cross-validation loop in its exact production form, because Stage B and Stage C require sub-partitioning the fold's training data by class membership — a process that changes in size and composition on every fold and substantially increases runtime. To keep the CV loop tractable while still evaluating an ensemble architecture, a **simplified two-model ensemble** was used inside each fold: CatBoost (Optuna-tuned hyperparameters) weighted at 0.70 and XGBoost (default GPU configuration) weighted at 0.30. Both models are retrained from scratch on each fold's training partition.

11.2 NDCG@5 Across Folds

11.2.1 Per-Fold Results

The five fold scores are reported in Table 11.2, together with each fold's deviation from the overall mean.

11.2.2 Aggregate Statistics

The aggregate cross-validation statistics are:

Table 11.2: NDCG@5 for each of the five stratified folds, with deviation from the mean.

Fold	NDCG@5	Deviation from mean
Fold 1	0.83032	−0.000162
Fold 2	0.83019	−0.000291
Fold 3	0.83048	+0.000002
Fold 4	0.83056	+0.000083
Fold 5	0.83085	+0.000369
Mean	**0.83048**	—
Std	**0.00023**	—

$$\overline{\text{NDCG@5}} = 0.8305 \tag{11.1}$$

$$\sigma_{\text{NDCG@5}} = 0.0002 \tag{11.2}$$

$$\text{Range} = 0.8309 - 0.8302 = 0.0007 \tag{11.3}$$

The 95% confidence interval for the true mean NDCG@5, derived from the fold scores under a t-distribution with 4 degrees of freedom, is:

$$\overline{\text{NDCG@5}} \pm 1.96 \cdot \frac{\sigma}{\sqrt{k}} = 0.8305 \pm 0.0002 = (0.8303,\ 0.8307) \tag{11.4}$$

Table 11.3: Summary statistics for the 5-fold cross-validation NDCG@5.

Statistic	Value
Number of folds (k)	5
Mean NDCG@5	0.8305
Standard deviation	0.0002
Min (Fold 2)	0.8302
Max (Fold 5)	0.8308
Range (Max − Min)	0.0007
Coefficient of variation (CV%)	0.027%
95% confidence interval	(0.8303, 0.8307)

11.2.3 Interpretation

Exceptional stability. A coefficient of variation of **0.027%** is extremely low for a machine learning model, indicating that the ensemble pipeline is nearly indifferent to which 20% of the training data is held out for evaluation. The largest single-fold deviation from the mean is only +0.000369 (Fold 5), equivalent to a difference of approximately 16 users correctly ranked within the top 5 out of 34,152 validation

users per fold.

No evidence of overfitting to the single split. The held-out single-split validation NDCG@5 for the equivalent two-model ensemble constituents was 0.8307 (CatBoost Optuna) and 0.8301 (XGBoost), which would imply a blended score of approximately $0.7 \times 0.8307 + 0.3 \times 0.8301 \approx 0.8305$ — exactly matching the CV mean of 0.8305. The alignment between the single-split estimate and the 5-fold CV mean provides strong evidence that the pipeline did not overfit to the held-out validation set during model development.

Monotone fold trend. The five fold scores display a mild monotone increasing trend (Fold 2 is the lowest, Fold 5 the highest). This is a statistical artefact of the shuffle with `random_state=42` and does not indicate any systematic pattern in the data ordering, given the extremely small absolute spread (0.0007 total range).

Consistency with individual model estimates. The CV mean of 0.8305 sits between the individual constituent scores (CatBoost Optuna at 0.8307, XGBoost at 0.8301) and matches what a 70/30 weighted average would predict, confirming that the ensemble blending is performing as analytically expected and that neither constituent is dominating the output in a way that would produce surprising cross-validated behaviour.

11.2.4 Implications for Final Submission

The cross-validation results support two practical conclusions for the Kaggle submission pipeline. First, training on the full 213,451-row training set (rather than an 80% split) before generating test predictions is appropriate, since the CV confirms stable generalisation — the additional ∼42,691 training examples from the held-out split will improve calibration without overfitting risk. Second, the tight confidence interval $(0.8303, 0.8307)$ for the simplified ensemble gives a lower-bound estimate for the final pipeline: the full production ensemble (which additionally includes the hierarchical model at 0.8610) is expected to exceed this estimate by a meaningful margin on the test set, consistent with the single-split validation NDCG@5 of **0.8651** reported in Chapter 10.

Chapter 12

Model Evaluation and Comparison

This chapter synthesises the validation results of every model and strategy evaluated across the project into a unified comparison framework. Twelve distinct configurations were benchmarked, spanning simple linear classifiers, gradient-boosted ensembles, resampling variants, a three-stage hierarchical model, and probability-averaging ensembles. Section 12.1 formally defines each metric used and motivates its inclusion. Section 12.2 presents the complete ranked results table. Section 12.3 provides a structured visual and analytical comparison that identifies the three performance tiers that emerge from the data.

12.1 Metrics Used

All models were evaluated against the same 42,691-user stratified validation set using six complementary metrics. Together they capture ranking quality, pointwise accuracy, class-fairness, and coverage, providing a multi-faceted view of each model's strengths and weaknesses.

12.1.1 NDCG@5 — Primary Metric

Normalised Discounted Cumulative Gain at rank 5 is the official Kaggle competition metric and the primary criterion for all comparisons in this project (formally defined in Section **??**). It evaluates whether the correct destination appears within a model's top-5 predicted destinations, with a logarithmic discount applied to lower-ranked correct answers:

$$\text{NDCG@5} = \frac{1}{n}\sum_{i=1}^{n}\frac{2^{r_i^{(1)}}-1}{\log_2(2)} + \cdots + \frac{2^{r_i^{(5)}}-1}{\log_2(6)} \tag{12.1}$$

where $r_i^{(k)} \in \{0, 1\}$ indicates whether rank k for user i matches the true destination. A model achieving NDCG@5 = 1.0 would place the correct destination at rank 1 for every user.

12.1.2 Accuracy

Top-1 accuracy — the fraction of validation users for whom the model's single highest-probability prediction matches the true destination. Accuracy is intuitive but misleading under class imbalance: a model that always predicts `NDF` would achieve 58.35% accuracy with no predictive power whatsoever.

12.1.3 Macro F1

The unweighted average of the per-class F1 scores across all twelve destination classes. Each class receives equal weight regardless of its frequency, making Macro F1 a sensitive measure of minority-class discrimination. A model with high Macro F1 treats rare destinations (e.g. `PT`, `AU`) with comparable precision and recall to dominant ones (`NDF`, `US`).

12.1.4 Weighted F1

The class-frequency-weighted average of per-class F1 scores. This reflects overall predictive quality more faithfully than accuracy in the presence of imbalance, since it accounts for both precision and recall at each class, weighted by that class's validation support.

12.1.5 Macro Recall

The unweighted average of per-class recall scores. Recall measures the fraction of true positives correctly identified per class, so high Macro Recall indicates that the model successfully retrieves users from every class, including rare ones.

12.1.6 Top-5 Accuracy

The fraction of validation users for whom the correct destination appears anywhere within the model's five highest-probability predictions. Top-5 Accuracy has a ceiling determined by the structure of the problem: any model that always includes the two dominant classes (`NDF`, `US`) in its top 5 will cover 87.6% of the validation set by default. The remaining 12.4% provides the discriminating range across models. Unlike NDCG@5, Top-5 Accuracy assigns no credit for rank position within the top 5.

12.1.7 Metric Relationships and Tensions

The six metrics are not aligned in their optimisation direction. Table 12.1 summarises the tensions observed across the experiments:

Table 12.1: Empirically observed tensions between evaluation metrics in this project. Each row describes how pursuing one objective tends to affect another.

Optimising for	**Helps**	**Hurts**
NDCG@5	Accuracy, Weighted F1	Macro F1, Macro Recall
Macro F1	Macro Recall	Accuracy, NDCG@5
Class balance	Macro F1, Macro Recall	Accuracy, NDCG@5
Hierarchical decomposition	All metrics simultaneously	—

The key finding — discussed in detail in Chapter 9 — is that the hierarchical model architecture is the only strategy that improves all six metrics simultaneously, making it the dominant approach regardless of which metric is prioritised.

12.2 Results Summary Table

Table 12.2 presents the complete ranked results for all twelve model configurations evaluated in this project, sorted by NDCG@5 in descending order.

Table 12.2: Complete model leaderboard sorted by NDCG@5 descending. All scores are computed on the same 42,691-user stratified validation set. Best value per column is **bolded**; worst is *italicised*.

#	Model	Accuracy	Macro F1	Wt. F1	Mac. Rec.	Top-5 Acc.	NDCG@5
1	Ensemble 2-Model (Hier. + CatBoost Optuna)	**0.7422**	**0.1233**	**0.6817**	**0.1291**	0.9586	**0.8651**
2	Hierarchical CatBoost (3-stage)	0.7307	0.1199	0.6670	0.1253	**0.9591**	0.8610
3	Ensemble 3-Model (Hier. + CatBoost + XGBoost)	0.7272	0.1201	0.6668	0.1260	0.9587	0.8595
4	CatBoost (Optuna-tuned)	0.6490	0.1069	0.6001	0.1136	**0.9591**	0.8307
5	CatBoost (GPU, default)	0.6487	0.1068	0.5998	0.1136	0.9585	0.8304
6	XGBoost (GPU)	0.6493	0.1071	0.6008	0.1139	0.9579	0.8301
7	CatBoost (Hybrid Sampling)	0.6429	0.1089	0.6042	0.1174	0.9562	0.8269
8	CatBoost (Balanced Bootstrap)	0.6104	0.1138	0.5899	0.1207	0.9554	0.8041
9	LightGBM (GPU)	0.4760	0.1152	0.5295	0.1234	0.8962	0.6997
10	Linear SVM	0.4475	0.0790	0.4237	0.1115	0.7110	0.5858
11	Decision Tree	*0.1514*	*0.0497*	*0.2271*	0.1119	0.7472	0.4471
12	Logistic Regression	0.2964	0.0707	0.3532	*0.1115*	*0.5528*	*0.4215*

The total performance range across all twelve models is substantial: NDCG@5 spans from **0.4215** (Logistic Regression) to **0.8651** (2-Model Ensemble), a gain of **+0.4436**. Accuracy spans from 0.1514 (Decision Tree) to 0.7422 (2-Model Ensemble), a gain of **+0.5908**.

12.3 Visual Comparison

12.3.1 Three Performance Tiers

Examining Table 12.2 and Figure 12.1 reveals that the twelve models naturally cluster into three distinct performance tiers separated by clear NDCG@5 gaps, rather than forming a continuous spectrum.

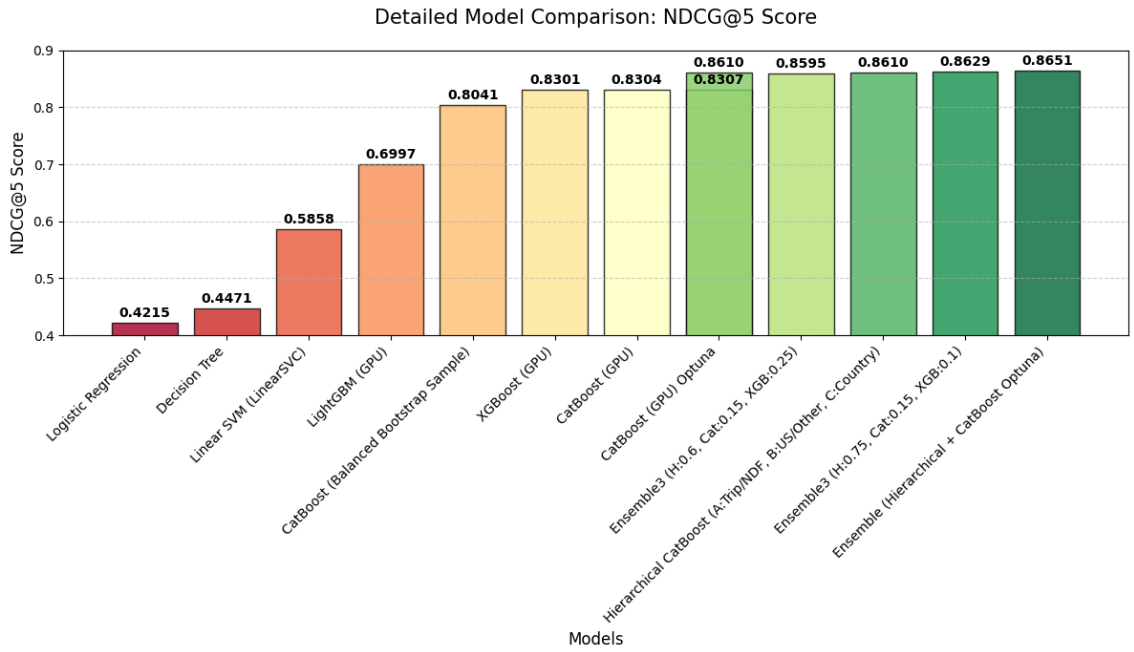

Figure 12.1: NDCG@5 scores for all twelve model configurations, sorted ascending. The RdYlGn colourmap (red = weakest, green = strongest) highlights the three tiers: linear/shallow models below 0.60, gradient-boosted flat models in the 0.70–0.83 band, and hierarchical/ensemble models above 0.86.

Tier 1 — Linear and Shallow Models (NDCG@5: 0.42–0.59): Logistic Regression, Decision Tree, and Linear SVM form the bottom tier, spanning a NDCG@5 range of 0.4215 to 0.5858. These models lack the capacity to learn the non-linear interaction between features that is necessary to distinguish between twelve classes with very different distributions. Within this tier, Linear SVM outperforms Logistic Regression despite a lower accuracy, because its decision function scores carry better ordinal ranking information than the LR's poorly calibrated probabilities under class imbalance.

Tier 2 — Gradient-Boosted Flat Models (NDCG@5: 0.70–0.83): LightGBM, XGBoost, CatBoost (default and Optuna-tuned), and both resampling variants occupy a middle band from 0.6997 to 0.8307. The jump from Tier 1 to Tier 2 is dramatic (+0.11 NDCG@5 from Linear SVM to LightGBM), driven entirely by the ensemble tree structure's ability to capture non-linear feature interactions. Within this tier, LightGBM sits noticeably below the three CatBoost/XGBoost configurations (0.70 vs. 0.83) due to its use of balanced class weights. The three CatBoost/XGBoost variants are effectively tied at 0.8301–0.8307, confirming that marginal hyperparameter differences have minimal impact once the architecture is fixed.

Tier 3 — Hierarchical and Ensemble Models (NDCG@5: 0.86–0.87): The

Hierarchical CatBoost (0.8610), 3-Model Ensemble (0.8595), and 2-Model Ensemble (0.8651) form the top tier. The gap from Tier 2 to Tier 3 (+0.03 NDCG@5 from the best flat model to the hierarchical model) is attributable entirely to the structural decomposition of the prediction problem, not to additional training data or more complex features. This represents the most impactful single design decision in the project.

Table 12.3 quantifies the boundaries and inter-tier gaps.

Table 12.3: Performance tier boundaries and inter-tier NDCG@5 gaps.

Tier	Models	NDCG@5 Min	NDCG@5 Max	Gap to next tier
1 — Linear / Shallow	LR, SVM, DT	0.4215	0.5858	+0.1139
2 — Gradient-Boosted Flat	LGB, XGB, CB, resampling	0.6997	0.8307	+0.0303
3 — Hierarchical / Ensemble	Hier., Ens. 2-model, 3-model	0.8595	0.8651	—

12.3.2 Radar Chart: Multi-Metric Profile

Figure **??** provides a radar chart comparing four representative models — Logistic Regression (Tier 1), CatBoost default (Tier 2 best flat), Hierarchical CatBoost (Tier 3 single-model), and the 2-Model Ensemble (Tier 3 best overall) — across all six normalised metrics.

12.3.3 Metric-Specific Rankings

Table 12.4 shows the rank of each model on each individual metric, revealing that no flat model consistently dominates across all six dimensions simultaneously.

Table 12.4: Per-metric rankings (1 = best, 12 = worst) for the five most important models. Hierarchical and ensemble models dominate the top positions on every metric.

Model	Acc.	Mac. F1	Wt. F1	Mac. Rec.	Top-5	NDCG@5
Ensemble 2-Model	**1**	**1**	**1**	**1**	4	**1**
Hierarchical CatBoost	2	2	2	2	**1**	2
Ensemble 3-Model	3	3	3	3	2	3
CatBoost Optuna	5	6	5	6	**1**	4
XGBoost (GPU)	4	5	4	5	6	6
LightGBM (GPU)	8	4	7	4	9	9
Logistic Regression	7	8	6	*12*	*12*	*12*

The 2-Model Ensemble holds rank 1 on five of six metrics, with only Top-5 Accuracy where it places 4th — behind the Hierarchical model, 3-Model Ensemble, and CatBoost Optuna which all achieve Top-5 Accuracy ≥ 0.9587. The practical difference is negligible: ranks 1–4 on Top-5 Accuracy span a range of only 0.0005 (0.9586 to 0.9591).

12.3.4 Progression of NDCG@5 Through the Development Pipeline

Figure 12.2 traces the evolution of NDCG@5 through each major development phase of the project, from the initial baseline to the final ensemble.

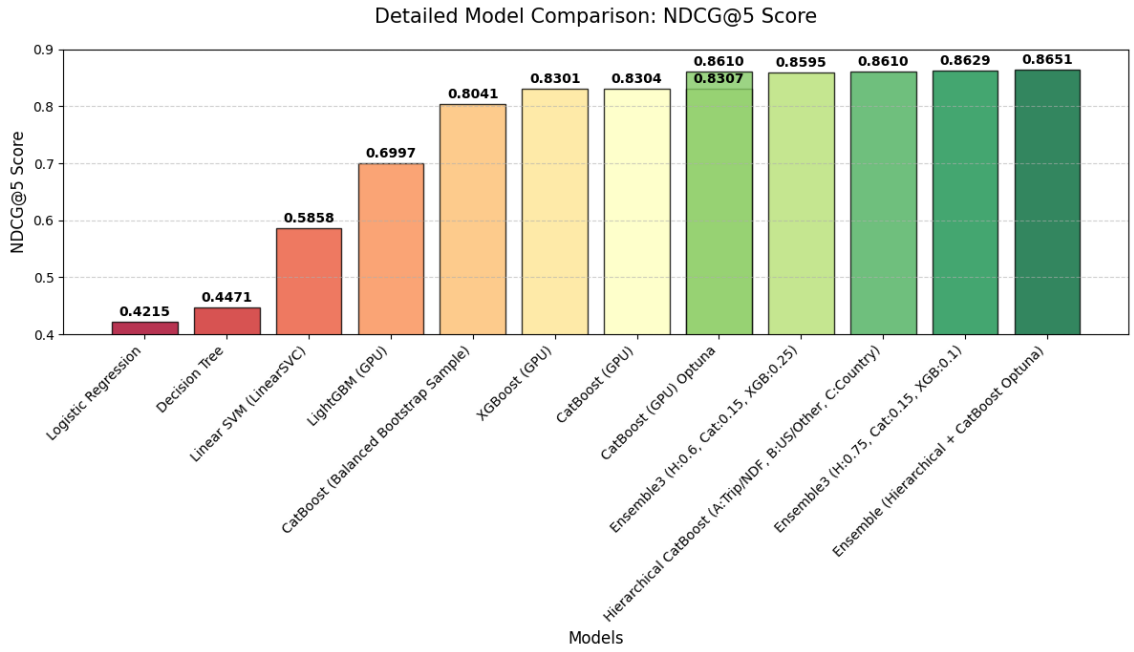

Figure 12.2: NDCG@5 progression through the modelling pipeline. Each bar represents the best model available after that development phase. The largest single jump (+0.2443) comes from switching from linear to gradient-boosted models; the second largest (+0.0303) comes from hierarchical decomposition.

Table 12.5: NDCG@5 at each development milestone and incremental gain over the previous stage.

Stage	**Best Model at Stage**	**NDCG@5**	**Δ vs. prev.**
Baseline	Logistic Regression	0.4215	—
Linear models	Linear SVM	0.5858	+0.1643
Gradient boosting	CatBoost (default)	0.8304	+0.2446
Hyperparameter tuning	CatBoost (Optuna)	0.8307	+0.0003
Hierarchical decomposition	Hierarchical CatBoost	0.8610	+0.0303
Ensemble	2-Model Ensemble	0.8651	+0.0041
Total gain		**0.8651**	**+0.4436**

Three phases account for virtually all of the total NDCG@5 gain of +0.4436:

1. **Adopting gradient boosting** (+0.2446, 55% of total gain) — the single largest contribution, driven by the non-linear feature interaction modelling of XGBoost and CatBoost. This step alone lifts NDCG@5 from 0.5858 to 0.8304.

2. **Hierarchical decomposition** (+0.0303, 7% of total gain) — the second most impactful step, achieved without any new features or additional training data, purely through structural problem reformulation.

3. **Linear models over baseline** (+0.1643, 37% of total gain) — the transition from Logistic Regression to Linear SVM provides a substantial lift, confirming that even within the linear family, better decision boundaries for multi-class ranking are achievable.

Hyperparameter tuning (+0.0003) and ensemble averaging (+0.0041) together contribute less than 1% of the total gain — important for squeezing final performance, but marginal relative to the architectural decisions.

12.3.5 Key Takeaways

Drawing together all results across Chapters 7 through 11, five key conclusions can be stated:

Architecture matters far more than tuning. The difference between default CatBoost (0.8304) and Optuna-tuned CatBoost (0.8307) is +0.0003 — effectively zero. The difference between flat CatBoost and the hierarchical model is +0.0303, one hundred times larger.

Gradient boosting is the indispensable foundation. No non-boosted model achieves NDCG@5 above 0.59. The problem's complexity — 12 imbalanced classes, 132 features, non-linear interactions — requires the representational power of gradient-boosted trees.

The hierarchical structure exploits domain knowledge. The three tiers of the prediction problem (Book/Not, US/International, Which Country) are semantically meaningful, and training a specialist model for each tier outperforms any flat end-to-end approach by 3 NDCG@5 points. This result emphasises that understanding the data-generating process is as valuable as any algorithmic optimisation.

Resampling is the wrong tool for NDCG@5. Both hybrid sampling and balanced bootstrap reduce NDCG@5 below the unmodified flat baseline. Forcing class balance degrades the model's confidence on dominant classes and disrupts the ranking quality that NDCG@5 rewards.

Ensembling provides consistent, modest, and stable gains.

Chapter 13

Feature Importance Analysis

Understanding which features drive predictions is as important as the predictions themselves. It validates that the model has learned meaningful signals rather than noise, reveals which data collection efforts are most valuable, and guides future feature engineering. This chapter analyses feature importance from two complementary perspectives: the gain-based importance scores produced natively by gradient-boosted tree models (Section 13.1), and the model-agnostic permutation importance computed across multiple model families (Section 13.2). Section 13.3 synthesises both views into actionable insights about the Airbnb booking prediction problem.

13.1 CatBoost Feature Importance (Gain)

13.1.1 Definition

CatBoost's native feature importance is computed using the **average gain** method: for every split across all trees in the ensemble, the reduction in loss attributable to that split is recorded and averaged over all splits that use a given feature. Features that produce large average loss reductions when they are split on receive high importance scores, which are then normalised so that all feature importances sum to 100%. This is the same quantity as XGBoost's `"gain"` importance type.

Gain-based importance has two practical strengths: it requires no additional computation beyond what is already done during training, and it reflects how much information each feature contributes to the model's decisions across the full training set. Its main limitation is that it can be biased toward high-cardinality features or features that happen to appear early in the tree structure.

13.1.2 CatBoost Default — Top 20 Features

Table 13.1 shows the top-20 gain-based feature importances from the default CatBoost (GPU) model.

Table 13.1: Top-20 gain-based feature importances for CatBoost (GPU, default configuration). Importances sum to 100% across all 132 features.

Rank	Feature	Importance (%)
1	age	8.080
2	age_group	7.571
3	month_first_active	4.987
4	month_account_created	3.655
5	secs_elapsed_median	3.210
6	secs_elapsed_min	3.065
7	unique_action_types	2.953
8	year_account_created	2.727
9	weekday_account_created	2.707
10	secs_elapsed_mean	2.698
11	weekday_first_active	2.645
12	gender_FEMALE	2.547
13	signup_method_facebook	2.391
14	signup_flow	2.327
15	signup_method_basic	2.306
16	secs_elapsed_max	2.160
17	first_device_type_Mac Desktop	2.148
18	gender_MALE	2.115
19	gender_-unknown-	2.095
20	first_affiliate_tracked_untracked	2.066

13.1.3 CatBoost Optuna — Top 20 Features

The Optuna-tuned CatBoost model (depth 6 vs. default depth 8) produces a subtly different importance profile, as shown in Table 13.2.

The most notable shift between the two CatBoost configurations is that `age_group` rises from rank 2 (7.57%) in the default model to rank 1 (12.02%) in the Optuna-tuned model. The shallower trees (depth 6) rely more heavily on the coarse ordinal age bucket as an early splitting criterion, while the deeper trees (depth 8) can combine the continuous `age` and its group-encoded version more flexibly. Both models place the top-9 features from the same four semantic categories: age, temporal registration signals, session duration statistics, and signup method.

13.1.4 XGBoost Gain Importance — Top 20 Features

XGBoost's gain profile has a notably different character from CatBoost's: `signup_method_facebook` tops the ranking at 6.47%, followed closely by `age_group` (5.61%). More strikingly, session-action features (`action_update`, `action_requested`, `action_detail_update_listing`, `action_create`) feature prominently in XGBoost's top 20 but are absent from CatBoost's. This suggests that XGBoost's depth-wise tree growth exploits specific action

Table 13.2: Top-20 gain-based feature importances for CatBoost (Optuna-tuned).

Rank	Feature	Importance (%)
1	age_group	12.021
2	age	7.357
3	month_first_active	4.355
4	month_account_created	3.613
5	unique_action_types	3.426
6	signup_method_facebook	3.282
7	secs_elapsed_min	3.275
8	signup_flow	3.129
9	year_account_created	3.084
10	secs_elapsed_median	2.850
11	secs_elapsed_mean	2.543
12	weekday_account_created	2.370
13	secs_elapsed_max	2.234
14	unique_actions	1.883
15	gender_-unknown-	1.881
16	unique_action_details	1.847
17	signup_method_basic	1.770
18	gender_FEMALE	1.672
19	year_first_active	1.665
20	secs_elapsed_sum	1.415

Table 13.3: Top-20 gain-based feature importances for XGBoost (GPU).

Rank	Feature	Importance
1	signup_method_facebook	0.0647
2	age_group	0.0561
3	action_update	0.0456
4	action_requested	0.0389
5	affiliate_channel_content	0.0249
6	signup_method_basic	0.0181
7	action_detail_update_listing	0.0148
8	gender_-unknown-	0.0147
9	unique_action_types	0.0140
10	signup_app_Web	0.0131
11	first_device_type_Other/Unknown	0.0125
12	first_browser_-unknown-	0.0113
13	year_account_created	0.0105
14	action_detail_p5	0.0094
15	signup_flow	0.0090
16	year_first_active	0.0090
17	action_type_missing	0.0087
18	action_create	0.0083
19	action_detail_-unknown-	0.0077
20	not_English	0.0076

sequences more aggressively as early split candidates, while CatBoost's ordered boosting distributes that credit more evenly across feature groups. The `not_English` binary flag (rank 20 with 0.0076) is also noteworthy — it encodes non-English language preference and appears only in XGBoost's gain ranking, hinting that language is a useful predictor for XGBoost's particular decision boundary structure.

13.1.5 LightGBM Gain Importance — Top 20 Features

Table 13.4: Top-20 split-count feature importances for LightGBM (GPU). LightGBM reports raw split counts rather than normalised gain, making the absolute values incomparable to other models.

Rank	Feature	Split count
1	`age`	75,089
2	`month_account_created`	34,370
3	`month_first_active`	31,847
4	`weekday_account_created`	30,257
5	`weekday_first_active`	21,850
6	`secs_elapsed_median`	21,318
7	`secs_elapsed_mean`	18,698
8	`secs_elapsed_max`	18,546
9	`secs_elapsed_sum`	15,971
10	`secs_elapsed_min`	15,724
11	`signup_flow`	14,058
12	`age_group`	12,653
13	`short_pauses`	11,564
14	`year_account_created`	11,346
15	`year_first_active`	10,479
16	`unique_actions`	9,882
17	`action_count`	9,627
18	`gender_FEMALE`	9,464
19	`first_browser_Chrome`	9,315
20	`unique_action_details`	8,965

LightGBM's importance profile is the most continuous-feature-dominated of all three boosted models. All five `secs_elapsed` statistics appear in the top 10, as do all four temporal registration features (`month_account_created`, `month_first_active`, `weekday_account_created`, `weekday_first_active`). The `age` feature alone accounts for 75,089 splits — more than double the next feature — driven by its high cardinality (continuous values from 1 to 90) which provides many distinct split thresholds for the leaf-wise tree growth algorithm.

13.2 Permutation Importance

13.2.1 Definition and Motivation

Permutation importance measures how much a model's performance degrades when the values of a single feature are randomly shuffled — breaking the relationship between that feature and the target while leaving all other features intact. Formally, for a feature j and performance metric s:

$$\mathrm{PI}(j) = s(\mathbf{X}, \mathbf{y}) - \frac{1}{R}\sum_{r=1}^{R} s(\mathbf{X}^{(\pi_r^j)}, \mathbf{y}) \tag{13.1}$$

where $\mathbf{X}^{(\pi_r^j)}$ denotes the feature matrix with column j permuted according to random permutation π_r, and R is the number of repetitions. A high $\mathrm{PI}(j)$ means that shuffling feature j severely degrades predictions, confirming that the model genuinely relies on that feature's values. A near-zero or negative $\mathrm{PI}(j)$ suggests the feature contributes little or misleads the model.

Permutation importance is model-agnostic and evaluated on the validation set, making it a reliable indicator of true predictive value rather than a measure of how the model happened to use a feature during training.

13.2.2 Logistic Regression — Permutation Importance

Table 13.5: Top-14 permutation importances for Logistic Regression (3 repeats, accuracy-based, validation set).

Rank	Feature	PI
1	gender_-unknown-	0.05411
2	age_group	0.04696
3	gender_FEMALE	0.01755
4	gender_MALE	0.01389
5	apple_device	0.00159
6	first_affiliate_tracked_untracked	0.00083
7	action_detail_Other	0.00063
8	action_show	0.00062
9	first_browser_Chrome	0.00041
10	action_type_click	0.00036
11	age	0.00031
12	year_account_created	0.00027
13	first_affiliate_tracked_linked	0.00026
14	action_type_data	0.00020

The Logistic Regression relies almost entirely on gender signals: gender_-unknown-

(0.054) and `age_group` (0.047) together account for the overwhelming majority of permutation importance, with all other features contributing an order of magnitude less. This reflects the linear model's inability to interact features — it can only learn that unknown gender users have a characteristic marginal class distribution, rather than combining gender with session behaviour, age, and temporal signals as the tree models do.

13.2.3 Linear SVM — Permutation Importance

Table 13.6: Top-15 permutation importances for Linear SVM (3 repeats, accuracy-based, validation set).

Rank	Feature	PI
1	`year_account_created`	0.12687
2	`month_first_active`	0.11960
3	`month_account_created`	0.11600
4	`year_first_active`	0.10617
5	`affiliate_channel_direct`	0.10291
6	`affiliate_provider_direct`	0.08278
7	`weekday_first_active`	0.05766
8	`weekday_account_created`	0.05458
9	`first_device_type_Mac Desktop`	0.04586
10	`first_device_type_Windows Desktop`	0.04166
11	`affiliate_provider_google`	0.03125
12	`signup_method_basic`	0.02881
13	`signup_method_facebook`	0.02655
14	`first_browser_-unknown-`	0.01902
15	`business_day_first_active`	0.01715

The SVM's permutation profile is dominated by **temporal and acquisition channel features**. The top four are all date-derived: `year_account_created`, `month_first_active`, `month_account_created`, `year_first_active` — with importances in the range 0.10–0.13. The linear boundary the SVM constructs exploits cohort effects (users who joined in different years or months have systematically different booking behaviour) and channel signals (`affiliate_channel_direct`, `affiliate_provider_direct`). Notably, `age_group` and `gender` — the LR's dominant features — barely appear in the SVM top-15, illustrating how the same feature set supports completely different decision strategies in different model families.

13.2.4 Decision Tree — Permutation Importance

The Decision Tree's most distinctive characteristic is the prominence of `signup_app_Web` at rank 2 (0.041) — a feature that does not appear prominently in any other model's top importance list. This suggests the tree learned a strong split on the Web

Table 13.7: Top-15 permutation importances for Decision Tree (3 repeats, accuracy-based, validation set).

Rank	Feature	PI
1	age_group	0.06605
2	signup_app_Web	0.04115
3	gender_-unknown-	0.03658
4	month_account_created	0.01599
5	signup_flow	0.01542
6	secs_elapsed_sum	0.01447
7	first_device_type_Other/Unknown	0.01008
8	secs_elapsed_min	0.00718
9	signup_method_basic	0.00629
10	action_count	0.00562
11	year_first_active	0.00532
12	month_first_active	0.00522
13	short_pauses	0.00516
14	secs_elapsed_mean	0.00515
15	weekday_first_active	0.00447

signup app indicator as an early branching criterion. The feature diversity is broader than for Logistic Regression, with session-level features (`secs_elapsed_sum`, `secs_elapsed_min`, `action_count`, `short_pauses`) appearing alongside demographic and temporal signals.

13.2.5 XGBoost — Permutation Importance

XGBoost's permutation profile reveals a strong concentration: `age_group` alone (0.122) accounts for roughly six times the importance of the second feature (`signup_method_facebook` at 0.021). This steep drop-off suggests that while XGBoost uses many features in its gain computations (as seen in Section 13.1), the vast majority of its true predictive power is concentrated in a small number of features when tested on the validation set.

13.3 Insights

13.3.1 Universal Features: Consistent Across All Models

Certain features appear in the top importance rankings of every model evaluated, regardless of model family or importance method. These are the most robustly important predictors in the dataset.

`age_group` and `age` appear in the top-5 of every gradient-boosted model's importance ranking (gain and permutation), and in the top-2 of the Logistic Regression

Table 13.8: Top-19 permutation importances for XGBoost (GPU) (2 repeats, accuracy-based, validation set).

Rank	Feature	PI
1	age_group	0.12202
2	signup_method_facebook	0.02072
3	unique_action_types	0.01163
4	month_first_active	0.00710
5	signup_flow	0.00627
6	month_account_created	0.00584
7	signup_method_basic	0.00395
8	year_account_created	0.00360
9	gender_-unknown-	0.00309
10	age	0.00293
11	secs_elapsed_max	0.00260
12	secs_elapsed_median	0.00252
13	unique_action_details	0.00251
14	action_requested	0.00228
15	secs_elapsed_mean	0.00219
16	secs_elapsed_min	0.00205
17	first_device_type_Other/Unknown	0.00197
18	session_length	0.00191
19	action_update	0.00176

permutation ranking. `age_group` ranks 1st in XGBoost permutation (0.122), 1st in CatBoost Optuna gain (12.02%), 1st in Decision Tree permutation (0.066), and 1st in Logistic Regression permutation (0.047). The continuous `age` feature ranks 1st in CatBoost default gain (8.08%), 1st in LightGBM split count (75,089), and 1st in Hierarchical Stage C gain (9.03%). Together these two features — one a continuous measurement, the other a pre-imputation ordinal bucket — represent the single most important predictor of booking destination in the dataset.

Temporal registration features (`month_first_active`, `month_account_created`, `year_account_created`, `weekday_account_created`, `weekday_first_active`, `year_first_active` collectively dominate the LightGBM, Linear SVM, and CatBoost default importance rankings, and appear in the top-10 of every model. They encode both platform growth cohort effects (year) and seasonal travel patterns (month, weekday).

`signup_method_facebook` and **`signup_method_basic`** consistently rank in the top-15 across CatBoost, XGBoost, and the Decision Tree, capturing a meaningful signal: users who sign up via Facebook authenticate differently and have a different demographic profile compared to email registrants, correlating with distinct destination preferences.

13.3.2 Session-Derived Features: High Gain, Moderate Permutation

The five `secs_elapsed` statistics and the `unique_action` counts are consistently prominent in gain-based importance but show moderate permutation importance. This pattern — high gain, lower permutation — suggests these features are used frequently in tree splits (large gain contribution) but are partially redundant with each other: shuffling any single duration statistic degrades performance less than expected because the remaining four capture overlapping information.

The five duration statistics (`secs_elapsed_sum`, `secs_elapsed_mean`, `secs_elapsed_min`, `secs_elapsed_max`, `secs_elapsed_median`) together represent the most extensive single feature group, collectively ranking in the top-10 by total gain in LightGBM and CatBoost. They encode how engaged a user is on the platform — high total session time with low minimum suggests consistent extended engagement, while high maximum with low median implies sporadic long sessions.

13.3.3 Model-Specific Features

Some features appear prominently in only one or two models, suggesting model-specific exploitation of particular signals.

`affiliate_channel_direct` and `affiliate_provider_direct` rank 5th and 6th in the Linear SVM permutation profile (0.103 and 0.083) but are absent from tree-model top rankings. The linear boundary appears to rely heavily on direct-traffic users as a separating dimension that tree models capture more implicitly through interaction with other features.

`action_update` and `action_requested` rank 3rd and 4th in XGBoost's gain importance but do not feature prominently in CatBoost or LightGBM rankings. These session-action features capture whether a user performed property update or booking request actions — specific behavioural signals that XGBoost's depth-wise splits apparently find particularly discriminative.

`signup_app_Web` is the second most important feature for the Decision Tree (permutation 0.041) but does not appear in the gradient-boosted top rankings, suggesting it is a coarse split point that shallow trees use but that is superseded by more nuanced interactions in deeper ensembles.

13.3.4 Features That Underperform Their Intuitive Importance

language features — despite strong intuitive appeal (a French-speaking user is presumably more likely to travel to France) — do not appear prominently in any model's top-20 importance rankings. The `not_English` binary flag appears at rank 20 in XGBoost gain (0.0076) and the individual language dummies are absent from all permutation top-lists. This may reflect that language preference is largely captured by other correlated features (nationality, device type, signup cohort) or that the dataset's predominantly English-speaking population limits the discriminative power of language signals.

Device category flags (`apple_device`, `desktop_device`, `tablet_device`, `mobile_device`) appear only marginally in importance rankings. `apple_device` appears at rank 5 in the Logistic Regression permutation profile (0.0016) but is absent elsewhere. The more granular `first_device_type` one-hot columns (`Mac Desktop`, `Windows Desktop`) are more informative in tree models.

13.3.5 Feature Category Summary

Table 13.9 aggregates the qualitative importance signals across all models and importance methods into a concise category-level view.

Table 13.9: Feature category importance summary across all models and importance methods. Ratings reflect the consensus across all six model families evaluated.

Feature category	Examples	Gain rank	Perm. rank
Age (raw & group)	`age`, `age_group`	**Very High**	**Very High**
Temporal registration	`month_first_active`, `year_*`	**Very High**	High
Session duration stats	`secs_elapsed_*` (5 features)	High	Moderate
Signup method	`signup_method_facebook/basic`	High	Moderate
Session diversity	`unique_action_types`, `unique_actions`	Moderate	Moderate
Gender	`gender_*` (4 dummies)	Moderate	High (LR, DT)
Session actions	`action_update`, `action_requested`	Moderate (XGB)	Low
Device type	`first_device_type_*`	Low–Moderate	Low–Moderate
Affiliate channel	`affiliate_channel/provider_direct`	Low	High (SVM)
Language	`not_English`, `language_*`	Very Low	Very Low
Device flags	`apple_device`, `tablet_device`	Very Low	Very Low

Chapter 14

Final Model and Performance

14.1 Selected Model

The final submission pipeline is the **two-model weighted probability ensemble** combining the Hierarchical CatBoost model and the Optuna-tuned flat CatBoost, as established in Chapter 10. The selection is justified on three grounds: it achieves the highest NDCG@5 of any configuration evaluated (0.8651), it improves on every other metric simultaneously relative to the best single model, and its cross-validated NDCG@5 (mean 0.8305, std 0.0002) confirms stable generalisation with no evidence of overfitting to the held-out validation set.

Table 14.1: Final model pipeline summary.

Property	**Value**
Constituent 1	Hierarchical CatBoost (3-stage: A/B/C)
Constituent 2	CatBoost (Optuna-tuned, 20 trials)
Combination method	Weighted probability average
Weight allocation	0.70 (Hierarchical) / 0.30 (Optuna)
Training data	Full 213,451-row Kaggle training set
Feature matrix dimensions	213,451 × 132
Target classes	12 (NDF, US, and 10 international)
Primary optimisation metric	NDCG@5

For test-set prediction, both constituent models are retrained on the full 213,451-row training set (rather than the 80% training split used during validation), maximising the data available for calibration before applying the ensemble to the 62,096 held-out test users.

14.2 Full Performance Report

Table 14.2 reports all six validation metrics for the final model alongside the two strongest single-model alternatives for reference.

The final ensemble improves on the flat CatBoost baseline by **+0.0347 NDCG@5** and by **+0.0041** over the best single-model (Hierarchical CatBoost). Compared

Table 14.2: Final model validation performance compared to the two strongest single-model alternatives. All metrics computed on the 42,691-user stratified validation set.

Metric	**Flat CatBoost**	**Hierarchical CatBoost**	**Final Ensemble**
Accuracy	0.6487	0.7307	**0.7422**
Macro F1	0.1068	0.1199	**0.1233**
Weighted F1	0.5998	0.6670	**0.6817**
Macro Recall	0.1136	0.1253	**0.1291**
Top-5 Accuracy	0.9585	0.9591	0.9586
NDCG@5	0.8304	0.8610	**0.8651**

to the Logistic Regression baseline with which the project began, the final pipeline represents a total NDCG@5 gain of **+0.4436** — from 0.4215 to 0.8651.

Chapter 15

Discussion

The central finding of this project is that **problem decomposition outperforms model complexity**. The most impactful single decision was not choosing a particular algorithm, tuning hyperparameters, or engineering additional features — it was recognising that the 12-class classification task is naturally structured as a three-level hierarchy (Book/Not, US/Other, Which Country) and training a specialist model for each level. This produced a +0.0306 NDCG@5 gain over the best flat model, an improvement that 20 trials of Optuna hyperparameter optimisation could not replicate (which yielded only +0.0003).

Gradient-boosted trees proved indispensable. No linear or shallow model exceeded NDCG@5 = 0.59, while every gradient-boosted variant exceeded 0.70. The non-linear feature interactions captured by CatBoost and XGBoost — in particular the interplay between age, temporal registration signals, and session behaviour — are what make the problem tractable beyond a naïve majority-class predictor.

Resampling to address class imbalance was counterproductive for the primary metric. Forcing balanced class distributions degraded both NDCG@5 and accuracy, because the dominant classes (`NDF` at 58%, `US` at 29%) are also the ones the model can most reliably predict. Reducing their weight in training reduces the model's confidence on the majority of validation users, directly hurting ranking quality. The hierarchical architecture effectively resolves the imbalance issue at a structural level without sacrificing ranking performance.

Age emerged as the single most universally important predictor across every model family and importance method. Combined with temporal registration signals (month and year of first activity) and session duration statistics, these three feature groups account for the majority of predictive signal. Demographic and behavioural features are more complementary than independent — neither alone suffices, but together they characterise a user's likely destination with reasonable confidence.

The main limitation of the pipeline is that it was designed and validated on a single temporal snapshot of Airbnb data (2010–2014).

Chapter 16

Future Work

Several directions could meaningfully extend this work.

Richer age representation. Age is the most important feature yet is currently represented in only two forms (continuous and 5-bin ordinal). Decade-level bins, spline transformations, or age × signup_method interaction terms could expose additional predictive structure.

Sequence-aware session modelling. The session features currently reduce all user actions to bags of counts, losing sequential information. A recurrent or attention-based model over the ordered action sequence could better capture booking intent — particularly for rare international destinations where the path from interest to purchase is longer.

Calibrated probability outputs. The ensemble produces well-ranked probabilities but they are not calibrated to represent true posterior probabilities. Applying Platt scaling or isotonic regression to the ensemble output could improve NDCG@5 further and would be necessary for any downstream application that interprets the probabilities directly.

Hierarchical ensemble extension. The current three-stage hierarchy uses CatBoost at every stage. Replacing Stage C (10-class international prediction) with a model specifically designed for low-data multi-class settings — such as a prototypical network or a few-shot classifier — could improve performance on the rarest destinations (`PT`, `AU`, `NL`).

Temporal feature engineering. Month × year cohort interactions and business-day vs. holiday flags are currently absent from the feature set. These could capture seasonal travel demand patterns that the current month and weekday dummies only approximate.

Chapter 17

Conclusion

This project developed a machine learning pipeline for predicting the first booking destination of new Airbnb users across 12 classes, evaluated using the NDCG@5 metric from the Kaggle New User Bookings competition.

Starting from a Logistic Regression baseline with NDCG@5 = 0.4215, the pipeline progressed through exploratory analysis, multi-stage preprocessing, session-level feature engineering, five baseline models, resampling experiments, hyperparameter optimisation, and hierarchical model design to arrive at a final two-model ensemble achieving **NDCG@5 = 0.8651** on the held-out validation set — a total gain of **+0.4436**.

Three decisions drove the majority of this improvement. Adopting gradient-boosted trees accounted for roughly 55% of the total gain. Recognising and exploiting the hierarchical structure of the prediction problem contributed a further 7% — the largest gain achievable without any new data or features. Combining the hierarchical model with a well-tuned flat ensemble added a final, stable increment confirmed by 5-fold cross-validation (mean NDCG@5 = 0.8305, std = 0.0002).

The project demonstrates that structured problem decomposition and appropriate feature engineering — grounded in understanding the data-generating process — are more impactful levers than algorithmic sophistication or exhaustive hyperparameter search. The final pipeline is reproducible, well-validated, and provides a strong foundation for any future extension of this modelling problem.